# KI für Audio: Stand der Technik, Methoden und Anwendungen in Audiosignalverarbeitung & -generierung

aus dem Studiengang angewandte Informatik

an der Dualen Hochschule Mannheim

von

**Orcun Yilmaz**

**Louis Ulrich**

27.03.2026

**Matrikelnummer, Kurs:** 6989258, TINF23AI2
9167392, TINF23AI2

**Betreuer an der DHBW:** Prof. Dr. Eckhard Kruse

# Selbstständigkeitserklärung

Gemäß Ziffer 1.1.13 der Anlage 1 zu §§ 3, 4 und 5 der Studien- und Prüfungsordnung für die Bachelorstudiengänge im Studienbereich Technik der Dualen Hochschule Baden- Württemberg vom 29.09.2017. Wir versichern hiermit, dass wir unsere Arbeit mit dem Thema:

**KI für Audio: Stand der Technik, Methoden und Anwendungen in Audiosignalverarbeitung & -generierung**

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt haben. Wir versichern zudem, dass alle eingereichten Fassungen übereinstimmen.

Mannheim, 27.03.2026

Orcun Yilmaz

Louis Ulrich

# Inhaltsverzeichnis

**1 Einleitung und Zielsetzung ... 1**
1.1 Motivation des Projekts ... 1
1.2 Problemstellung: Bild-zu-Musik-Generierung ... 2
1.3 Ziel der Arbeit und Abgrenzung ... 2

**2 Theoretische Grundlagen ... 4**
2.1 Multimodale KI-Systeme ... 4
2.2 Bildbeschreibung mittels neuronaler Netze ... 5
2.3 Text-zu-Audio-Generierung ... 6

**3 Gesamtkonzept und Systemarchitektur ... 7**
3.1 Überblick über die Pipeline ... 7
3.2 Trennung von Aufgaben: Python vs. `C#` ... 9
3.3 Datenfluss zwischen den Komponenten ... 11
3.4 Ordnerstruktur und Projektaufbau ... 12

**4 Implementierung der Bild-zu-Text-Phase ... 15**
4.1 Auswahl und Einbindung des Caption-Modells ... 15
4.2 Verarbeitung von Bilddateien ... 17
4.3 Generierung und Speicherung der Captions ... 19

**5 Ableitung musikalischer Eigenschaften aus Text ... 22**
5.1 Motivation für hybride Umsetzung der Zuordnung ... 22
5.2 Struktur des Mapping-Skripts ... 23
5.3 Ausgabe und Weiterverarbeitung ... 26

**6 Generierung von Musik aus den abgeleiteten Eigenschaften ... 29**
6.1 Auswahl des MusicGen-Modells ... 29
6.2 Erstellung der Musik ... 30

**7 Audioanalyse und Vergleich der generierten Musik ... 35**
7.1 Ziel der Analyse ... 35
7.2 Extrahierte Audio-Features ... 36
7.3 Vergleich mehrerer Versionen eines Bildes ... 39
7.4 Automatisch erzeugte Vergleichsberichte ... 43

**8 Orchestrierung der Pipeline mit `C#` ... 47**
8.1 Aufruf und Steuerung der Python-Skripte ... 47
8.2 Automatisierte Ausführung der gesamten Pipeline ... 51

**9 Entwicklung der grafischen Benutzeroberfläche ... 54**
9.1 Zielsetzung der Benutzeroberfläche ... 54
9.2 Auswahl von WPF und Projektintegration ... 55
9.3 Aufbau des UI-Layouts ... 56
9.4 Interaktion zwischen UI und Pipeline ... 58
9.5 Wiedergabe, Speicherung und Analyseanzeige ... 63

**10 Erkenntnisse ... 67**
10.1 Technische Probleme und Lösungen ... 67
10.2 Analyse der WAV-Varianten ... 68
10.3 Ergebnisse aus Beispieldurchläufen ... 69
10.3.1 Inputbilder ... 69
10.3.2 Zwischenergebnis 1: Caption ... 70
10.3.3 Zwischenergebnis 2: Mapping / Musikparameter ... 70
10.3.4 Zwischenergebnis 3: Analyse-Highlights ... 71
10.3.5 Audio-Outputs ... 72
10.4 Fazit und Ausblick ... 73

**Literatur ... a**

# Abbildungsverzeichnis

**Abbildung 1 Architekturdiagramm der Image-to-Music-Pipeline** ..... **8**

**Abbildung 2 Datenflussdiagramm der vierstufigen Verarbeitungskette** ..... **11**

**Abbildung 3 Benutzeroberfläche der WPF-Anwendung** ..... **57**

**Abbildung 4 Die Input Bilder** ..... **69**

# Codeverzeichnis

**Listing 1** **Ordnerstruktur** ..... 14
**Listing 2** **caption.py Ausschnitt 1** ..... 17
**Listing 3** **caption.py Ausschnitt 2** ..... 17
**Listing 4** **caption.py Ausschnitt 3** ..... 18
**Listing 5** **caption.py Ausschnitt 4** ..... 19
**Listing 6** **caption.py Ausschnitt 5** ..... 19
**Listing 7** **caption.py Ausschnitt 6** ..... 20
**Listing 8** **caption.py Ausschnitt 7** ..... 20
**Listing 9** **caption.py Ausschnitt 8** ..... 21
**Listing 10** **map_caption.py Ausschnitt 1** ..... 24
**Listing 11** **map_caption.py Ausschnitt 2** ..... 24
**Listing 12** **map_caption.py Ausschnitt 3** ..... 24
**Listing 13** **map_caption.py Ausschnitt 4** ..... 25
**Listing 14** **map_caption.py Ausschnitt 5** ..... 25
**Listing 15** **map_caption.py Ausschnitt 6** ..... 26
**Listing 16** **map_caption.py Ausschnitt 7** ..... 26
**Listing 17** **beispiel Ausgabe von map_caption.py** ..... 28
**Listing 18** **musicgen.py Ausschnitt 1** ..... 31
**Listing 19** **musicgen.py Ausschnitt 2** ..... 31
**Listing 20** **musicgen.py Ausschnitt 3** ..... 32
**Listing 21** **musicgen.py Ausschnitt 4** ..... 32
**Listing 22** **musicgen.py Ausschnitt 5** ..... 32

**Listing 23 musicgen.py Ausschnitt 6** ..... 33
**Listing 24 musicgen.py Ausschnitt 7** ..... 33
**Listing 25 musicgen.py Ausschnitt 8** ..... 33
**Listing 26 musicgen.py Ausschnitt 9** ..... 34
**Listing 27 analyze.py Ausschnitt 1** ..... 36
**Listing 28 analyze.py Ausschnitt 2** ..... 37
**Listing 29 analyze.py Ausschnitt 3** ..... 37
**Listing 30 analyze.py Ausschnitt 4** ..... 38
**Listing 31 analyze.py Ausschnitt 5** ..... 38
**Listing 32 analyze.py Ausschnitt 6** ..... 38
**Listing 33 analyze.py Ausschnitt 7** ..... 39
**Listing 34 analyze.py Ausschnitt 8** ..... 39
**Listing 35 analyze.py Ausschnitt 9** ..... 40
**Listing 36 analyze.py Ausschnitt 10** ..... 40
**Listing 37 analyze.py Ausschnitt 11** ..... 40
**Listing 38 analyze.py Ausschnitt 12** ..... 41
**Listing 39 analyze.py Ausschnitt 13** ..... 41
**Listing 40 analyze.py Ausschnitt 14** ..... 41
**Listing 41 analyze.py Ausschnitt 15** ..... 42
**Listing 42 analyze.py Ausschnitt 16** ..... 42
**Listing 43 analyze.py Ausschnitt 17** ..... 43
**Listing 44 analyze.py Ausschnitt 18** ..... 44
**Listing 45 Beispiel csv** ..... 44
**Listing 46 analyze.py Ausschnitt 19** ..... 44
**Listing 47 analyze.py Ausschnitt 20** ..... 44

**Listing 48 analyze.py Ausschnitt 21** ..... 45
**Listing 49 Beispiel Mark-Down** ..... 45
**Listing 50 analyze.py Ausschnitt 22** ..... 46
**Listing 51 analyze.py Ausschnitt 23** ..... 46
**Listing 52 analyze.py Ausschnitt 24** ..... 46
**Listing 53 Program.cs Ausschnitt 1** ..... 47
**Listing 54 Program.cs Ausschnitt 2** ..... 48
**Listing 55 Program.cs Ausschnitt 3** ..... 48
**Listing 56 Program.cs Ausschnitt 4** ..... 48
**Listing 57 Program.cs Ausschnitt 5** ..... 49
**Listing 58 Program.cs Ausschnitt 6** ..... 49
**Listing 59 Program.cs Ausschnitt 7** ..... 50
**Listing 60 Program.cs Ausschnitt 8** ..... 51
**Listing 61 Program.cs Ausschnitt 9** ..... 52
**Listing 62 Program.cs Ausschnitt 10** ..... 52
**Listing 63 Program.cs Ausschnitt 11** ..... 52
**Listing 64 Program.cs Ausschnitt 12** ..... 53
**Listing 65 MainWindow.xaml Ausschnitt 1** ..... 56
**Listing 66 MainWindow.xaml Ausschnitt 2** ..... 57
**Listing 67 MainWindow.xaml Ausschnitt 3** ..... 58
**Listing 68 MainWindow.xaml Ausschnitt 4** ..... 58
**Listing 69 MainWindow.xaml.cs Ausschnitt 1** ..... 59
**Listing 70 MainWindow.xaml.cs Ausschnitt 2** ..... 59
**Listing 71 MainWindow.xaml.cs Ausschnitt 3** ..... 59
**Listing 72 MainWindow.xaml.cs Ausschnitt 4** ..... 60

**Listing 73 MainWindow.xaml.cs Ausschnitt 5** ..... **60**

**Listing 74 MainWindow.xaml.cs Ausschnitt 6** ..... **60**

**Listing 75 MainWindow.xaml.cs Ausschnitt 7** ..... **60**

**Listing 76 MainWindow.xaml.cs Ausschnitt 8** ..... **61**

**Listing 77 MainWindow.xaml.cs Ausschnitt 9** ..... **62**

**Listing 78 MainWindow.xaml.cs Ausschnitt 10** ..... **62**

**Listing 79 MainWindow.xaml.cs Ausschnitt 11** ..... **63**

**Listing 80 MainWindow.xaml.cs Ausschnitt 12** ..... **63**

**Listing 81 MainWindow.xaml.cs Ausschnitt 13** ..... **64**

**Listing 82 MainWindow.xaml.cs Ausschnitt 14** ..... **64**

**Listing 83 MainWindow.xaml.cs Ausschnitt 15** ..... **65**

**Listing 84 MainWindow.xaml.cs Ausschnitt 16** ..... **65**

# Abkürzungsverzeichnis

| | |
|---|---|
| **BART** | Bidirectional and Auto-Regressive Transformers |
| **BLIP** | Bootstrapping Language-Image Pre-training |
| **CSV** | Comma-Separated Values |
| **JSON** | JavaScript Object Notation |
| **KI** | Künstliche Intelligenz |
| **MFCC** | Mel-Frequency Cepstral Coefficients |
| **PIL** | Python Imaging Library |
| **RMS** | Root Mean Square |
| **UI** | User Interface |
| **WAV** | Waveform Audio File Format |
| **WPF** | Windows Presentation Foundation |

# Glossar

| | |
|---|---|
| **Mel-Frequency-Cepstral-Coefficients** | Kompakte Koeffizienten, die das Spektrum eines Audiosignals auf der Mel-Skala (menschenähnliche Frequenzwahrnehmung) darstellen; Standardmerkmal für Sprach- und Musikanalyse |
| **Open-Source-Modell** | Ein Open-Source-Modell ist ein Software- oder KI-Modell, dessen Quellcode öffentlich zugänglich ist und von Dritten eingesehen, genutzt, verändert und weiterverbreitet werden kann. |
| **librosa** | Bibliothek für Audio und Musik Analyse in Python |

# 1 Einleitung und Zielsetzung

## 1.1 Motivation des Projekts

Die Motivation des Projekts ergibt sich aus dem wachsenden Interesse an Systemen, die unterschiedliche Formen menschlicher Wahrnehmung miteinander verbinden. Die Idee, aus einem Bild automatisch Musik zu erzeugen, verbindet visuelle Eindrücke mit auditiver Interpretation und greift damit einen kreativen menschlichen Prozess auf. Bilder lösen häufig Stimmungen, Assoziationen und Emotionen aus, die sich nicht nur sprachlich, sondern auch musikalisch ausdrücken lassen. Die Übertragung dieser subjektiven Wahrnehmung auf ein technisches System stellt eine besonders spannende Herausforderung dar und eröffnet neue Perspektiven auf den Einsatz moderner KI-Technologien.

Bild-zu-Musik-Generierung ist deshalb interessant, weil sie über klassische Anwendungsfälle der Bild oder Textverarbeitung hinausgeht. Während viele Systeme darauf abzielen, Inhalte zu erkennen, zu klassifizieren oder möglichst exakt zu beschreiben, bewegt sich dieses Projekt bewusst im Spannungsfeld zwischen Interpretation und Kreativität. Musik besitzt keine eindeutige objektive Bedeutung, sondern wird individuell wahrgenommen. Ein Bild kann daher auf sehr unterschiedliche Weise musikalisch interpretiert werden. Genau diese Mehrdeutigkeit macht das Problem interessant, da sie zeigt, dass KI nicht nur analytische Aufgaben übernehmen kann, sondern auch kreative Prozesse nachahmt oder erweitert.

Darüber hinaus bietet das Projekt die Möglichkeit, unterschiedliche Programmiersprachen und Werkzeuge sinnvoll zu kombinieren. Die bewusste Trennung zwischen KI-naher Verarbeitung und klassischer Softwareentwicklung unterstützt das Ziel, praxisrelevante Kompetenzen aufzubauen. Die Motivation liegt dabei nicht nur im funktionierenden Endergebnis, sondern auch in der nachvollziehbaren Darstellung des Weges dorthin. Damit wird der Entwicklungsprozess zu einem wichtigen Teil der Arbeit und leistet einen entscheidenden Beitrag zum Erkenntnisgewinn.

Insgesamt entsteht die Motivation des Projekts aus dem Zusammenspiel von technischem Interesse, kreativer Neugier und akademischem Anspruch. Die Bild-zu-Musik-Generierung

dient dabei als konkretes und anschauliches Beispiel, um zu untersuchen, wie moderne KI-Systeme gestaltet, kombiniert und analysiert werden können, ohne den Anspruch zu erheben, menschliche Kreativität vollständig zu ersetzen.

## 1.2 Problemstellung: Bild-zu-Musik-Generierung

Die Problemstellung der Bild-zu-Musik-Generierung liegt in der grundlegenden Schwierigkeit, visuelle Informationen in eine akustische Form zu überführen, für die es keine eindeutigen oder objektiven Zuordnungsregeln gibt. Während ein Bild relativ klar beschrieben oder klassifiziert werden kann, ist Musik stark von subjektiver Wahrnehmung, kulturellem Kontext und emotionaler Interpretation geprägt. Ein und dasselbe Bild kann bei verschiedenen Betrachtern völlig unterschiedliche musikalische Assoziationen hervorrufen. Diese Mehrdeutigkeit macht die Aufgabe besonders anspruchsvoll und unterscheidet sie deutlich von klassischen Problemen der Bild oder Textverarbeitung.

Ein Problem dabei ist, dass sich Bildinhalt und Musik nicht direkt in ihrer Bedeutung entsprechen. Ob ein Bild eher als ruhig oder dynamisch, als hell oder düster wahrgenommen wird, hängt von individuellen Interpretationen ab. Diese Eigenschaften lassen sich nicht eindeutig aus den visuellen Merkmalen eines Bildes ableiten. Für ein technisches System bedeutet dies, dass jede Zuordnung zwischen Bild und Musik zwangsläufig eine Interpretation darstellt und nicht als objektiv korrekt oder falsch bewertet werden kann. Die Bild-zu-Musik-Generierung bewegt sich damit im Spannungsfeld zwischen Analyse und Kreativität.

Die Problemstellung der Bild-zu-Musik-Generierung ist daher nicht auf eine einzelne technische Herausforderung reduzierbar. Sie ergibt sich aus dem Zusammenspiel von Unsicherheit, Kreativität und Mehrdeutigkeit, das sowohl auf konzeptioneller als auch auf technischer Ebene wirkt. Diese Eigenschaften machen das Problem besonders anspruchsvoll, aber zugleich auch besonders geeignet, um die Möglichkeiten und Grenzen moderner KI-Systeme zu untersuchen und kritisch zu reflektieren.

## 1.3 Ziel der Arbeit und Abgrenzung

Das Ziel der Arbeit besteht darin, ein funktionierendes und nachvollziehbares System zur Bild-zu-Musik-Generierung zu entwerfen, zu implementieren und zu analysieren. Im Mittelpunkt steht dabei nicht die Erzeugung eines möglichst hochwertigen oder künstlerisch perfekten

Musikstücks, sondern die systematische Untersuchung eines mehrstufigen Prozesses, der visuelle Informationen in musikalische Ausgaben überführt. Die Arbeit verfolgt somit einen klar praxisorientierten Ansatz, bei dem Architektur, Designentscheidungen und das Verhalten der einzelnen Komponenten im Vordergrund stehen.

Ein wichtiger Aspekt ist die Analyse der erzeugten Ergebnisse. Anstatt ein einzelnes Ergebnis zu präsentieren, wird bewusst mit mehreren Musikversionen pro Bild gearbeitet. Ziel ist es, Unterschiede und Gemeinsamkeiten zwischen diesen Versionen sichtbar zu machen und Rückschlüsse auf die Stabilität und Variabilität der Generierung zu ziehen. Die Arbeit möchte damit zeigen, wie sich kreative KI-Systeme verhalten, wenn sie wiederholt mit identischen Eingaben konfrontiert werden, und wie sich dieses Verhalten untersuchen lässt.

Gleichzeitig erfolgt eine klare Abgrenzung der Zielsetzung. Es ist nicht Ziel der Arbeit, eigene KI Modelle zu trainieren oder bestehende Modelle in ihrer Architektur zu optimieren. Die verwendeten Modelle stammen aus bestehenden Bibliotheken und werden als vortrainierte Open-Source-Modell eingesetzt. Der Fokus liegt auf ihrer Anwendung und Analyse, nicht auf ihrer Entwicklung.

Ebenso ist es nicht Ziel, eine objektive Bewertung der musikalischen Qualität der erzeugten Ergebnisse vorzunehmen. Musik wird als subjektives Medium verstanden, dessen Qualität nicht eindeutig messbar ist. Die Arbeit erhebt daher keinen Anspruch darauf, zu beurteilen, ob die generierte Musik gut oder schlecht ist. Stattdessen werden messbare Eigenschaften analysiert und interpretiert, um Unterschiede zwischen den Versionen aufzuzeigen und das Verhalten des Systems zu untersuchen.

Auch die Entwicklung eines produktionsreifen Systems ist nicht Gegenstand der Arbeit. Aspekte wie umfassende Optimierung, Plattformunabhängigkeit oder langfristige Wartbarkeit stehen nicht im Fokus.

Insgesamt zielt die Arbeit darauf ab, ein realistisches und gut erklärbares Beispiel für die Nutzung moderner KI-Systeme im kreativen Kontext zu liefern. Durch die klare Abgrenzung der Ziele und Nicht-Ziele wird der Fokus auf die wesentlichen Fragestellungen gelenkt und ein Rahmen geschaffen, in dem die Bild-zu-Musik-Generierung als Untersuchungsgegenstand sinnvoll und strukturiert betrachtet werden kann.

# 2 Theoretische Grundlagen

## 2.1 Multimodale KI-Systeme

Multimodale KI-Systeme sind Systeme der Künstlichen Intelligenz, die Informationen aus mehreren unterschiedlichen Datenmodalitäten gleichzeitig verarbeiten und miteinander verknüpfen. Eine Modalität bezeichnet dabei eine bestimmte Form von Datenrepräsentation, beispielsweise Text, Bild, Audio oder Video. Während klassische KI-Modelle meist nur auf eine einzelne Modalität spezialisiert sind (z. B. Bildklassifikation oder Sprachverarbeitung), integrieren multimodale Systeme mehrere Eingabeformen in einem gemeinsamen Modell.

Ziel multimodaler KI ist es, Zusammenhänge zwischen verschiedenen Datenarten zu erkennen und sie in einer gemeinsamen Bedeutungsrepräsentation zusammenzuführen. Dadurch können Systeme beispielsweise ein Bild analysieren und eine passende Textbeschreibung generieren oder aus einer textuellen Beschreibung passende Audioinhalte erzeugen. Die Kombination mehrerer Modalitäten ermöglicht ein tieferes Kontextverständnis, da Informationen aus verschiedenen Quellen einander ergänzen und disambiguieren können.

Im Kontext der Bildverarbeitung und Audiogenerierung bedeutet dies konkret: Ein Bild wird zunächst durch ein visuelles Modell analysiert, das relevante Merkmale wie Objekte, Stimmungen oder Szenenstrukturen extrahiert. Diese Informationen können anschließend in eine sprachliche oder abstrakte Repräsentation überführt werden. Auf dieser Basis kann ein weiteres Modell musikalische Eigenschaften wie Tempo, Instrumentierung oder Stimmung ableiten und daraus Audioinhalte generieren. Die multimodale Verknüpfung bildet somit die Grundlage für kreative Anwendungen wie Bild-zu-Musik-Transformationen.

Multimodale KI-Systeme gewinnen zunehmend an Bedeutung, da sie der menschlichen Wahrnehmung näherkommen, welche ebenfalls verschiedene Sinneseindrücke integriert. Durch Fortschritte im Deep Learning, insbesondere durch Transformer-Architekturen, ist es möglich geworden, gemeinsame Repräsentationsräume für unterschiedliche Modalitäten zu lernen und flexibel zwischen ihnen zu übersetzen.

## 2.2 Bildbeschreibung mittels neuronaler Netze

Die automatische Bildbeschreibung durch neuronale Netze, bekannt als Image Captioning, gehört zu den grundlegenden Verfahren moderner multimodaler KI-System. Ziel dieser Verfahren ist es, visuelle Inhalte automatisch in eine sprachliche Beschreibung zu überführen, die für Menschen verständlich ist und gleichzeitig von weiteren Systemen verarbeitet werden kann. Bildbeschreibungsmodelle fungieren damit als Schnittstelle zwischen visueller Wahrnehmung und symbolischer Repräsentation und ermöglichen es, Bilder in nachgelagerte textbasierte Prozesse einzubinden.

Grundsätzlich basieren solche Modelle auf neuronalen Netzen, die visuelle Merkmale aus einem Bild extrahieren und diese mit sprachlichen Strukturen verknüpfen. Während ein Teil des Modells für die Analyse des Bildinhalts zuständig ist, übernimmt ein weiterer Teil die Generierung der textuellen Beschreibung. Diese Kombination erlaubt es dem System, Objekte, Szenen oder einfache Beziehungen innerhalb eines Bildes zu erkennen und in vollständige Sätze zu überführen. Das Ergebnis ist keine reine Aufzählung von Bildmerkmalen, sondern eine zusammenhängende Beschreibung, die den Bildinhalt auf abstrakter Ebene zusammenfasst.

Gleichzeitig besitzen solche Modelle klare Grenzen. Die erzeugten Beschreibungen sind immer eine Interpretation des Bildes und keine objektive Abbildung aller enthaltenen Informationen. Feinere Details, subtile Stimmungen oder kontextuelle Hintergründe werden häufig nicht erfasst oder nur implizit angedeutet. Zudem neigen Bildbeschreibungsmodelle dazu, plausible, aber nicht zwingend korrekte Inhalte zu ergänzen. Dieses Verhalten entsteht aus der statistischen Natur der Modelle, die versuchen, wahrscheinliche Beschreibungen zu erzeugen, auch wenn bestimmte Informationen im Bild nicht eindeutig vorhanden sind.

Ein weiteres charakteristisches Merkmal ist die Abhängigkeit vom Trainingsmaterial. Die Wortwahl und Struktur der erzeugten Captions spiegeln die Daten wider, auf denen das Modell trainiert wurde. Dadurch können bestimmte Perspektiven oder kulturelle Annahmen reproduziert werden, während andere Aspekte unterrepräsentiert bleiben. Für das System bedeutet dies, dass die erzeugten Beschreibungen nicht neutral sind, sondern immer im Kontext der gelernten Muster stehen.

## 2.3 Text-zu-Audio-Generierung

Das grundlegende Prinzip von Text zu Audio Systemen ist, semantische Informationen aus dem Text zu extrahieren und diese in musikalische oder akustische Strukturen zu überführen. Der Text wird dabei nicht wortwörtlich in Klänge übersetzt, sondern dient als abstrakte Beschreibung eines gewünschten Ergebnisses. Begriffe wie ruhig, schnell oder düster werden vom Modell als Hinweise interpretiert, die Einfluss auf Rhythmus, Klangfarbe oder Intensität haben. Die konkrete Ausgestaltung dieser Hinweise erfolgt innerhalb des Modells und ist nicht vollständig deterministisch.

Im Kontext der Musikgenerierung kommen spezialisierte Modelle zum Einsatz, die auf große Mengen musikalischer Daten trainiert wurden. Diese Modelle lernen typische Strukturen, Harmonien und Klangmuster und können diese auf neue Texteingaben anwenden. MusicGen ist ein Beispiel für ein solches Modell, das speziell für die Generierung von Musik aus textuellen Beschreibungen entwickelt wurde. Auf hoher Ebene betrachtet kombiniert es ein Verständnis für sprachliche Konzepte mit einem generativen Modell für Audiosignale.[1]

# 3 Gesamtkonzept und Systemarchitektur

## 3.1 Überblick über die Pipeline

Der Überblick über die Pipeline bildet das konzeptionelle Fundament des gesamten Systems und beschreibt den vollständigen Ablauf von der visuellen Eingabe bis zur analytischen Auswertung der generierten Musik. Ziel der Pipeline ist es, aus einem einzelnen Bild mehrere musikalische Interpretationen abzuleiten und diese anschließend systematisch miteinander zu vergleichen. Die Pipeline ist dabei bewusst als lineare Abfolge klar voneinander abgegrenzter Verarbeitungsschritte gestaltet, um sowohl die technische Umsetzung als auch die spätere Analyse nachvollziehbar zu halten.

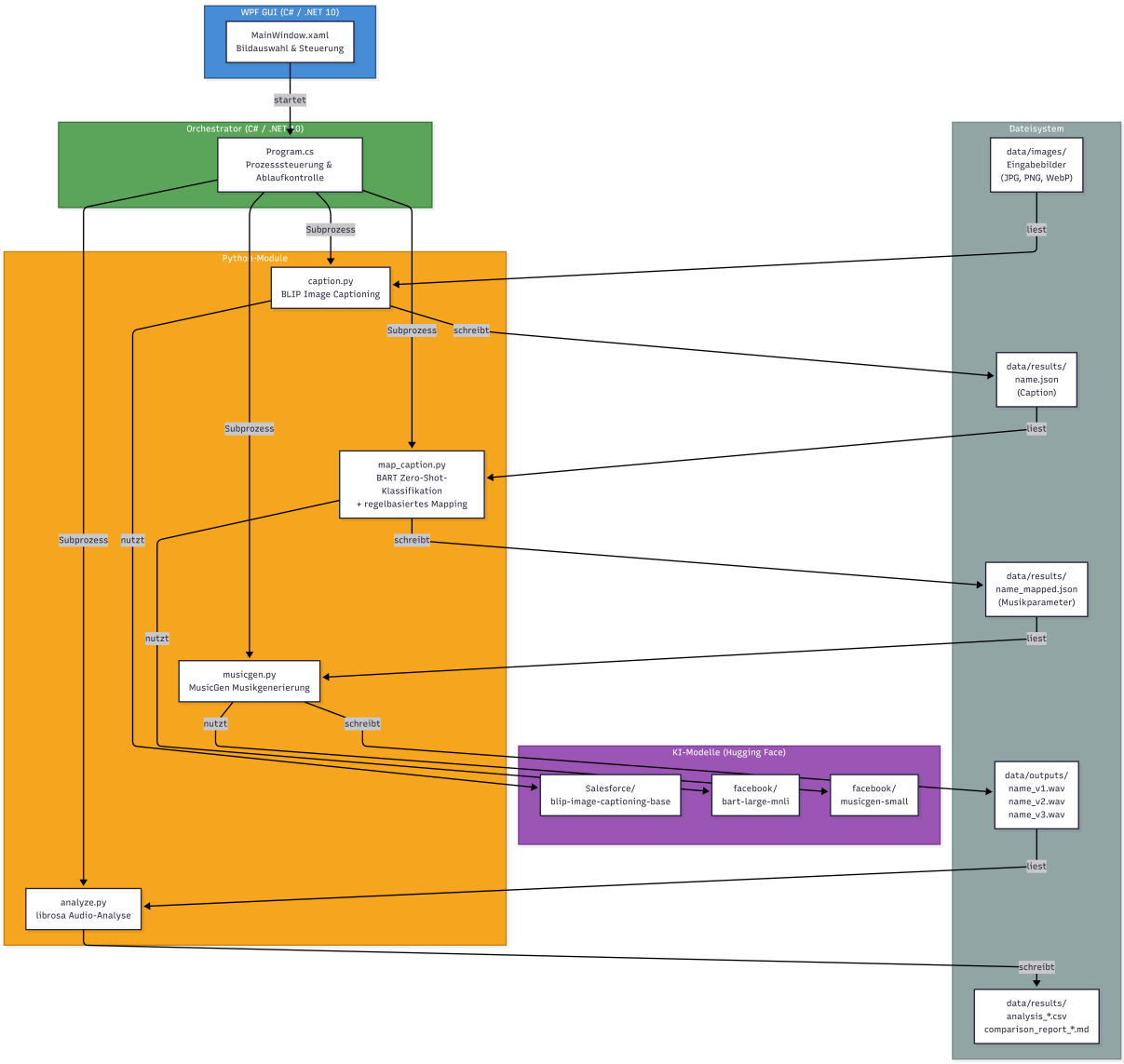


Abbildung 1 — Architekturdiagramm der Image-to-Music-Pipeline

Ausgangspunkt der Pipeline ist ein digitales Bild, das durch den Benutzer ausgewählt wird. Dieses Bild stellt die einzige externe Eingabe des Systems dar und fungiert als Ausgangsbasis für alle folgenden Verarbeitungsschritte. Da Bilder eine rein visuelle Informationsquelle darstellen, ist es notwendig, diese zunächst in eine für weitere Verarbeitungsschritte geeignete Repräsentation zu überführen. Zu diesem Zweck wird das Bild in eine textuelle Beschreibung übersetzt, die Inhalte und visuelle Merkmale in natürlicher Sprache zusammenfasst. Diese textuelle Beschreibung dient als semantische Verdichtung der Bildinformation und ermöglicht es, visuelle Eindrücke in einer Form abzulegen, die von nachfolgenden Modulen weiterverarbeitet werden kann.

Im Anschluss an die Bildbeschreibung erfolgt die Ableitung musikalischer Eigenschaften aus dem erzeugten Text. In diesem Schritt wird die sprachliche Beschreibung analysiert und in abstrakte musikalische Parameter überführt. Diese Parameter umfassen unter anderem eine angenommene Stimmung, ein musikalisches Genre sowie ein Zieltempo. Anstatt konkrete musikalische Inhalte wie Melodien oder Harmonien festzulegen, beschreiben diese Parameter lediglich den Rahmen, innerhalb dessen sich die spätere Musikgenerierung bewegt. Dadurch bleibt der kreative Spielraum erhalten, während gleichzeitig eine gewisse thematische Kohärenz zur Bildbeschreibung gewährleistet wird.

Die Musikgenerierung ist der nächste Schritt in der Pipeline. Auf Basis der zuvor abgeleiteten Parameter wird ein textueller Steuerinput erzeugt, der sowohl die musikalischen Eigenschaften als auch die ursprüngliche Bildbeschreibung in komprimierter Form enthält. Dieser Steuertext dient als Eingabe für ein generatives Musikmodell, das mithilfe von KI Verfahren Audiosignale erzeugt.

Den Abschluss der Pipeline bildet die automatische Analyse der erzeugten Musikdateien. In diesem Schritt werden aus den Audiosignalen verschiedene Merkmale extrahiert, die eine objektive Beschreibung der Musik ermöglichen. Diese Merkmale beziehen sich sowohl auf zeitliche als auch auf spektrale Eigenschaften der Audiosignale und dienen als Grundlage für einen systematischen Vergleich der einzelnen Versionen. Die Analyseergebnisse werden in tabellarischer Form gespeichert und zusätzlich in einem lesbaren Bericht zusammengefasst, der die Unterschiede zwischen den Musikversionen eines Bildes strukturiert darstellt.

## 3.2 Trennung von Aufgaben: Python vs. `C#`

Die Trennung der Aufgaben zwischen Python und `C#` stellt ein grundlegendes Architekturprinzip des entwickelten Systems dar und ist maßgeblich für dessen Struktur, Verständlichkeit und Erweiterbarkeit verantwortlich. Die Entscheidung für den parallelen Einsatz beider Programmiersprachen basiert nicht auf technischen Zwängen, sondern auf einer bewussten Abwägung ihrer jeweiligen Stärken in Bezug auf die Anforderungen der Bild zu Musik Pipeline. Da das Projekt sowohl Aspekte der KI Verarbeitung als auch klassische Softwarearchitektur und Benutzerinteraktion vereint, war es sinnvoll, diese Aufgabenbereiche klar voneinander abzugrenzen.

Python wird innerhalb des Systems für alle Aufgaben eingesetzt, die unmittelbar mit maschinellem Lernen, Audioverarbeitung und Datenanalyse verbunden sind. Die Sprache bietet eine Vielzahl etablierter Bibliotheken und Frameworks, die speziell für diese Anwendungsfälle entwickelt wurden. Vortrainierte Modelle für Bildbeschreibung, Textklassifikation und Musikgenerierung sind in Python häufig direkt verfügbar und lassen sich mit vergleichsweise geringem Implementierungsaufwand einbinden. Viele dieser Modelle werden als Open-Source-Modelle bereitgestellt, was eine transparente Nutzung und eine einfache Reproduzierbarkeit der Ergebnisse ermöglicht. Darüber hinaus erlaubt Python eine schnelle Iteration während der Entwicklungsphase, was insbesondere bei experimentellen Anpassungen von Parametern oder Modellkonfigurationen von Vorteil ist.

`C#` hingegen wird als übergeordnete Steuerungs- und Anwendungsschicht eingesetzt. Die Sprache eignet sich besonders für die Strukturierung komplexer Programmabläufe, für ein sauberes Fehlerhandling sowie für die Entwicklung grafischer Benutzeroberflächen. Hier fungiert `C#` als Steuerungseinheit für den gesamten Pipeline-Ablauf. Die einzelnen Python Skripte werden nicht isoliert ausgeführt, sondern gezielt von der `C#` Anwendung aufgerufen, überwacht und ausgewertet. Dadurch entsteht ein kontrollierter und reproduzierbarer Ablauf, bei dem jeder Verarbeitungsschritt klar definiert ist.

Darüber hinaus ermöglicht der Einsatz von `C#` eine robuste Integration einer grafischen Benutzeroberfläche. Während Python zwar grundsätzlich ebenfalls Möglichkeiten zur GUI-Entwicklung bietet, liegt die Stärke von `C#` insbesondere im Zusammenspiel mit etablierten Frameworks für Desktop-Anwendungen. Die Benutzeroberfläche dient in diesem Projekt nicht nur als Komfortfunktion, sondern als integraler Bestandteil des Systems, da sie den Zugang zur Pipeline vereinfacht und deren Funktionsweise anschaulich macht. Die Entscheidung, die GUI in `C#` umzusetzen, fügt sich somit nahtlos in das Gesamtkonzept der Architektur ein.

Ein weiterer Aspekt der Technologieentscheidung betrifft die Wartbarkeit und Erweiterbarkeit des Systems. Durch die klare Trennung der Aufgaben können einzelne Teile unabhängig voneinander weiterentwickelt werden.[2]

## 3.3 Datenfluss zwischen den Komponenten

Der Datenfluss zwischen den einzelnen Komponenten ist ein wesentliches Element der Systemarchitektur, da er die logisch getrennten Verarbeitungsschritte der Pipeline miteinander verbindet. Ziel des gewählten Ansatzes ist es, einen transparenten, nachvollziehbaren und reproduzierbaren Austausch von Informationen zu ermöglichen, ohne dass einzelne Komponenten direkt voneinander abhängig sind. Um dieses Ziel zu erreichen, erfolgt die Kommunikation zwischen allen Modulen ausschließlich über Dateien in standardisierten Formaten, wobei insbesondere JSON Dateien eine zentrale Rolle einnehmen.[3]

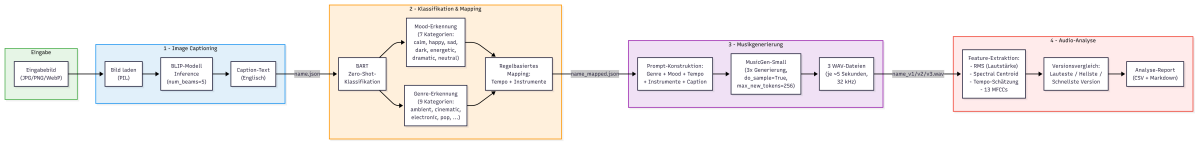


Abbildung 2 — Datenflussdiagramm der vierstufigen Verarbeitungskette

JSON eignet sich für den Datenaustausch innerhalb der Pipeline, da das Format sowohl für Maschinen als auch für Menschen leicht lesbar ist. Die strukturierte Darstellung erlaubt es, komplexe Informationen wie Bildbeschreibungen, abgeleitete musikalische Eigenschaften oder Analyseergebnisse übersichtlich abzulegen.

Jede Phase der Pipeline arbeitet nach dem gleichen Grundprinzip. Eine Komponente liest ihre benötigten Eingabedaten aus einer oder mehreren Dateien ein, verarbeitet diese Informationen und schreibt das Ergebnis in eine neue Datei. Auf diese Weise entstehen klar definierte Übergabepunkte zwischen den einzelnen Verarbeitungsschritten. So wird beispielsweise die vom Bildbeschreibungssystem erzeugte Caption in einer eigenen JSON Datei gespeichert, die anschließend als Eingabe für die Ableitung musikalischer Eigenschaften dient. Die daraus resultierenden Parameter werden wiederum in einer separaten Datei abgelegt, die als Grundlage für die Musikgenerierung fungiert.

Neben JSON Dateien werden für die Musikgenerierung und Analyse Audiodateien im WAV Format verwendet. Diese Dateien stellen das Ergebnis der Generierungsphase dar und werden ebenfalls eindeutig in den Datenfluss eingebettet.

Ein weiterer wichtiger Aspekt des Datenflusses ist die konsequente Versionierung der erzeugten Daten. Da pro Bild mehrere Musikversionen generiert werden, ist es notwendig, diese

eindeutig voneinander zu unterscheiden. Dies wird durch ein festgelegtes Benennungsschema erreicht, das sowohl den Bezug zum ursprünglichen Bild als auch eine Versionsnummer enthält.

Die Versionierung unterstützt zudem die Reproduzierbarkeit der Ergebnisse. Mehrere Durchläufe der Pipeline können parallel existieren, ohne sich gegenseitig zu überschreiben. Dies ist insbesondere dann relevant, wenn Parameter angepasst oder unterschiedliche Testbilder verwendet werden. Durch die Speicherung aller relevanten Zwischenergebnisse bleibt der vollständige Entwicklungs- und Analyseprozess dokumentiert und kann jederzeit nachvollzogen werden.

Der gewählte Ansatz des dateibasierten Datenflusses bringt darüber hinaus den Vorteil mit sich, dass einzelne Komponenten unabhängig voneinander weiterentwickelt oder ausgetauscht werden können.

## 3.4 Ordnerstruktur und Projektaufbau

Die Ordnerstruktur des Projekts orientiert sich direkt an der technischen Architektur der Pipeline und verfolgt das Ziel, Quellcode, Eingabedaten und generierte Ergebnisse klar voneinander zu trennen. Dadurch bleibt das Repository sowohl für Entwickler als auch für externe Betrachter nachvollziehbar und strukturiert.

Der Quellcode ist konsequent nach Technologien getrennt. Im Verzeichnis python befinden sich alle fachlichen Verarbeitungsschritte der Pipeline, also Bildbeschreibung, semantische Ableitung musikalischer Eigenschaften, Musikgenerierung sowie Audioanalyse. Die `C#`-Komponenten sind ebenfalls getrennt organisiert: Projektarbeit.Orchestrator übernimmt die Ausführung der Pipeline, während Projektarbeit.UI die grafische Benutzeroberfläche bereitstellt. Diese strukturelle Trennung spiegelt die inhaltliche Rollenverteilung zwischen Modelllogik und Orchestrierung wider.

Die Daten werden im Verzeichnis data verwaltet und sind dort klar gegliedert. Der Ordner images enthält ausschließlich Eingabebilder. Im Ordner outputs werden die generierten WAV-Dateien abgelegt, also die verschiedenen Musikversionen pro Bild. Das Verzeichnis results speichert sämtliche abgeleiteten Dateien, darunter JSON-Dateien mit gemappten Musikparametern, CSV-Dateien mit Analysewerten sowie automatisch erzeugte Markdown-Reports.

Damit ist eindeutig nachvollziehbar, welche Dateien Rohdaten, welche Zwischenstände und welche Analyseergebnisse darstellen.

Die Dateibenennung folgt einem konsistenten Schema, das den Bezug zum ursprünglichen Bild sowie eine Versionskennung enthält. Dadurch lassen sich zusammengehörige Dateien eindeutig zuordnen, ohne zusätzliche Metadaten zu benötigen. Diese Struktur unterstützt insbesondere die spätere Analyse und Vergleichbarkeit der Varianten.

Die folgende Abbildung zeigt die tatsächliche Projektstruktur des Repositories und verdeutlicht die beschriebene Trennung zwischen Code, Daten und Ergebnissen:

```
PROJEKTARBEIT
│
├── .venv
├── .vs
│
├── csharp
│   └── Projektarbeit.Orchestrator
│       ├── bin
│       ├── obj
│       ├── Program.cs
│       └── Projektarbeit.Orchestrator.csproj
│
├── data
│   ├── images          # Eingabebilder
│   ├── outputs         # generierte WAV-Dateien (v1, v2, v3)
│   └── results         # JSON-, CSV- und Markdown-Analyseergebnisse
│
│
├── Projektarbeit.UI
│   ├── bin
│   ├── obj
│   ├── App.xaml
│   ├── App.xaml.cs
│   ├── AssemblyInfo.cs
│   ├── MainWindow.xaml
│   ├── MainWindow.xaml.cs
│   ├── Projektarbeit.UI.csproj
│   └── Projektarbeit.UI.csproj.user
│
├── python
│   ├── analyze.py
│   ├── caption.py
│   ├── map_caption.py
│   ├── musicgen.py
│   └── requirements.txt
│
└── Projektarbeit.sln
```

Listing 1 — Ordnerstruktur

Insgesamt trägt die gewählte Ordnerstruktur wesentlich zur Wartbarkeit, Reproduzierbarkeit und Transparenz des Systems bei. Sie stellt sicher, dass sowohl die Pipeline als auch die Benutzeroberfläche eindeutig auf definierte Verzeichnisse zugreifen können, ohne dass es zu Vermischungen zwischen Eingabe- und Ergebnisdaten kommt.

# 4 Implementierung der Bild-zu-Text-Phase

## 4.1 Auswahl und Einbindung des Caption-Modells

Die Auswahl und Einbindung des Caption-Modells ist ein wichtiger Schritt in der Bild-zu-Text-Phase, da die Qualität der Bildbeschreibungen alle weiteren Verarbeitungsschritte beeinflusst. Da die Bildbeschreibung als semantische Grundlage für die Ableitung musikalischer Eigenschaften dient, musste ein Modell gewählt werden, das in der Lage ist, visuelle Inhalte möglichst zuverlässig und zugleich allgemeinverständlich in natürlicher Sprache zu beschreiben. Dabei lag der Fokus nicht auf maximaler Modellkomplexität, sondern auf einem ausgewogenen Verhältnis zwischen Beschreibungsqualität, Integrationsaufwand und Reproduzierbarkeit der Ergebnisse.

Für die Umsetzung wurde ein vortrainiertes Bildbeschreibungssystem aus dem Umfeld der KI eingesetzt, das auf dem Prinzip der multimodalen Verarbeitung basiert. Das gewählte Modell trägt die Bezeichnung BLIP und kombiniert visuelle und sprachliche Repräsentationen in einem gemeinsamen Modellansatz.[4] BLIP ist darauf ausgelegt, Bilder in frei formulierten Text zu übersetzen, ohne auf fest definierte Klassen oder Kategorien beschränkt zu sein.[4] Diese Eigenschaft ist für das vorliegende Projekt besonders relevant, da die Bildbeschreibungen nicht nur einzelne Objekte erkennen, sondern auch Szenen und implizite Stimmungen erfassen sollen.

Ein wesentlicher Grund für die Wahl von BLIP liegt in seiner hohen Flexibilität gegenüber unterschiedlichen Bildinhalten.[4] Da das System nicht auf einen spezifischen Bildtyp wie Landschaften oder Porträts zugeschnitten ist, kann es auf eine breite Palette von Motiven angewendet werden. Dies unterstützt das Ziel der Arbeit, ein generisches Bild zu Musik System zu entwickeln, das nicht auf vordefinierte Szenarien beschränkt ist. Gleichzeitig liefert BLIP in vielen Fällen natürlich klingende Beschreibungen, die sich gut für eine anschließende sprachliche Weiterverarbeitung eignen.

Ein weiterer entscheidender Faktor bei der Modellauswahl war die Verfügbarkeit als Open-Source-Modell.[4] Die Offenlegung des Modells und seiner Trainingsgrundlagen ermöglicht

eine transparente Nutzung und unterstützt die wissenschaftliche Nachvollziehbarkeit der Arbeit. Darüber hinaus erleichtert die offene Lizenzierung die Integration in eigene Projekte, ohne auf proprietäre Schnittstellen oder kostenpflichtige Dienste angewiesen zu sein. Diese Aspekte sind insbesondere im akademischen Kontext von Bedeutung, da sie eine reproduzierbare und überprüfbare Umsetzung ermöglichen.

Für die Integration des BLIP-Modells wurde die Plattform Hugging Face genutzt, die im KI-Bereich weit etabliert ist. Hugging Face stellt vortrainierte Modelle über eine einheitliche Programmierschnittstelle bereit und abstrahiert viele der technischen Details, die bei der direkten Nutzung neuronaler Netze anfallen würden. Durch diese Abstraktion konnte das Modell mit vergleichsweise geringem Implementierungsaufwand in die bestehende Pipeline integriert werden. Die Verwendung der bereitgestellten Bibliotheken erlaubt es, das Modell mit wenigen Codezeilen zu laden, zu konfigurieren und für die Generierung von Bildbeschreibungen zu verwenden.[5]

Die technische Einbindung des Modells folgt dabei einem klar strukturierten Ablauf. Zunächst wird das Bild in einem geeigneten Format geladen und für die Verarbeitung vorbereitet. Anschließend wird es gemeinsam mit den notwendigen Metadaten an das Modell übergeben, das daraus eine textuelle Beschreibung generiert. Diese Beschreibung wird als Zeichenkette ausgegeben und anschließend in einer JSON Datei gespeichert. Durch diese Speicherung wird die Bildbeschreibung dauerhaft verfügbar gemacht und kann unabhängig von der eigentlichen Modellausführung weiterverarbeitet oder analysiert werden.

Im praktischen Einsatz zeigte sich, dass die erzeugten Bildbeschreibungen eine gewisse Variabilität aufweisen können. Je nach Bildinhalt und Komplexität werden Details unterschiedlich gewichtet oder interpretiert. Diese Eigenschaft ist jedoch kein Nachteil, sondern ein integraler Bestandteil des Systems, da sie zur kreativen Offenheit der späteren Musikgenerierung beiträgt. Die Bildbeschreibung fungiert somit nicht als exakte Abbildung der visuellen Realität, sondern als interpretative Grundlage, auf der weitere Verarbeitungsschritte aufbauen.

Durch die Kombination aus einem leistungsfähigen Caption Modell, einer einfachen Einbindung über Hugging Face und einer klar definierten Ausgabeform fügt sich die Bildbeschreibung nahtlos in die Gesamtarchitektur der Pipeline ein. Sie bildet den ersten inhaltlichen

Verarbeitungsschritt des Systems und legt den Grundstein für alle nachfolgenden Phasen der Bild-zu-Musik-Generierung.

## 4.2 Verarbeitung von Bilddateien

Im Skript caption.py beginnt die Verarbeitung der Bilddatei nicht direkt mit der Erstellung der Bildbeschreibung, sondern zunächst mit einer sauberen technischen Vorbereitung der Eingabedaten. Ziel dieses Abschnitts ist es, sicherzustellen, dass ganz unterschiedliche Bilddateien, unabhängig von Format oder Eigenschaften, in eine einheitliche und verlässliche Form gebracht werden, bevor sie weiterverarbeitet werden.

Zu Beginn wird der Pfad zur Bilddatei über die Kommandozeile übergeben und in ein sogenanntes Path-Objekt umgewandelt:

```
if len(sys.argv) < 3:
  print("Usage: python caption.py <image_path> <output_json>")
  sys.exit(1)

image_path = Path(sys.argv[1])
output_path = Path(sys.argv[2])
```

Listing 2 — caption.py Ausschnitt 1

Hier wird zunächst geprüft, ob der Benutzer überhaupt einen Bildpfad und einen Speicherpfad angegeben hat. Falls nicht, wird das Programm mit einer kurzen Anleitung beendet. Das sorgt dafür, dass das Skript nicht mit unvollständigen Eingaben weiterläuft. Der übergebene Bildpfad wird anschließend mit Path(...) verarbeitet. Diese Klasse stammt aus pathlib und erleichtert den Umgang mit Dateipfaden, da sie betriebssystemunabhängig funktioniert und typische Fehler bei der Pfadverarbeitung reduziert.

Der eigentliche Verarbeitungsschritt beginnt mit dem Laden der Bilddatei:

```
image = Image.open(image_path).convert("RGB")
```

Listing 3 — caption.py Ausschnitt 2

Mit Image.open(image_path) wird das Bild aus dem Dateisystem geladen. Dabei nutzt das Skript die Bibliothek Pillow (PIL), die viele gängige Bildformate wie JPEG, PNG oder WEBP automatisch erkennt. Das bedeutet, dass das Skript keine eigene Logik zur Formatunterschei-

dung benötigt. Sollte die Datei beschädigt sein oder kein gültiges Bild enthalten, würde an dieser Stelle ein Fehler auftreten und das Programm abbrechen. Dadurch wird verhindert, dass ungültige Daten weiterverarbeitet werden.[6]

Direkt im Anschluss wird das Bild mit .convert(„RGB“) in einen einheitlichen Farbraum umgewandelt. Dieser Schritt ist besonders wichtig. Bilder können unterschiedlich gespeichert sein: manche enthalten einen Alphakanal, andere sind Graustufenbilder oder verwenden alternative Farbräume. Das verwendete Caption-System erwartet jedoch ein dreikanaliges RGB-Bild. Durch die Konvertierung wird sichergestellt, dass jedes Bild genau drei Farbkanäle besitzt, unabhängig davon, wie es ursprünglich gespeichert wurde. Ein Graustufenbild wird beispielsweise automatisch in ein RGB-Bild umgewandelt, indem die Helligkeitswerte auf drei Kanäle verteilt werden. Dadurch entsteht eine konsistente Struktur, die für alle Bilder gleich aussieht.[6]

Nachdem das Bild geladen und in RGB umgewandelt wurde, erfolgt die Vorbereitung für die Weiterverarbeitung:

```
1 inputs = processor(image, return_tensors="pt")
```

Listing 4 — caption.py Ausschnitt 3

Hier wird das Bild an den BLIPProcessor übergeben. Dieser Schritt wandelt das Bild in eine Form um, mit der das spätere KI-System arbeiten kann. Dabei passieren im Hintergrund mehrere Dinge automatisch. Das Bild wird auf die passende Größe gebracht, falls es zu groß oder zu klein ist. Außerdem werden die Pixelwerte so angepasst, dass sie im richtigen Zahlenbereich liegen. Schließlich wird das Bild in eine sogenannte Tensorstruktur umgewandelt. Ein Tensor kann man sich vereinfacht als mehrdimensionale Zahlenmatrix vorstellen, mit der moderne KI-Systeme rechnen können.[7]

Durch return_tensors=„pt“ wird festgelegt, dass diese Zahlenstruktur im Format von PyTorch ausgegeben wird. Das ist das Framework, das intern für die Berechnungen verwendet wird. [7], [8]

Zusammengefasst besteht die Bildverarbeitung in caption.py aus mehreren klar getrennten Schritten. Zunächst wird der Bildpfad sauber eingelesen und geprüft. Danach wird das Bild mit PIL geöffnet und in einen einheitlichen RGB-Farbraum umgewandelt. Anschließend wird es

durch den Processor automatisch skaliert, normalisiert und in eine rechenbare Zahlenstruktur überführt. Erst nach diesen vorbereitenden Maßnahmen ist das Bild technisch in einer Form, die eine stabile und zuverlässige Weiterverarbeitung ermöglicht. Die Bildverarbeitung stellt damit einen eigenständigen und wichtigen Teil der Architektur dar, da sie sicherstellt, dass unterschiedlichste Eingabebilder konsistent behandelt werden und keine unerwarteten Probleme im weiteren Verlauf der Pipeline entstehen.[5]

## 4.3 Generierung und Speicherung der Captions

Die Generierung und Speicherung der Captions bildet den abschließenden Schritt der Bild-zu-Text-Phase. Nachdem das Bild technisch vorbereitet und in eine geeignete interne Struktur überführt wurde, erfolgt die eigentliche Erstellung der textuellen Beschreibung sowie deren strukturierte Ablage auf dem Dateisystem. Hier spielt die erzeugte Caption eine wichtige Rolle, da sie die visuelle Eingabe mit den nachfolgenden Schritten der Pipeline verbindet.

Die Caption-Generierung basiert auf dem geladenen BLIP-Modell:

```
processor = BlipProcessor.from_pretrained("Salesforce/blip-image-captioning-base")
model = BlipForConditionalGeneration.from_pretrained("Salesforce/blip-image-captioning-base")
```

Listing 5 — caption.py Ausschnitt 4

Hier wird das vortrainierte Modell `"Salesforce/blip-image-captioning-base"` aus dem Hugging-Face-Repository geladen.[4] Der BLIPProcessor übernimmt dabei die Aufbereitung der Eingabedaten, während „BLIPForConditionalGeneration“ für die eigentliche Texterzeugung zuständig ist. Dieser Ladevorgang stellt sicher, dass das Modell mit allen notwendigen Gewichten und Konfigurationen bereitsteht.

Nachdem das Bild bereits durch den Processor vorbereitet wurde, wird es an das Modell übergeben:

```
output = model.generate(**inputs, max_length=80, num_beams=5)
```

Listing 6 — caption.py Ausschnitt 5

Die Methode `generate(...)` erzeugt automatisch eine textuelle Beschreibung. Dabei werden die zuvor berechneten Bildrepräsentationen verwendet. Der Parameter `max_length=80` begrenzt die maximale Länge der erzeugten Beschreibung auf 80 Tokens, wodurch übermäßig lange oder unkontrollierte Texte vermieden werden. Mit `num_beams=5` wird das sogenannte Beam Search Verfahren genutzt. Vereinfacht ausgedrückt bedeutet dies, dass das Modell mehrere mögliche Textvarianten parallel betrachtet und die wahrscheinlichste auswählt. Dadurch wird die Qualität der generierten Beschreibung in der Regel stabiler und konsistenter als bei einer rein zufälligen Texterzeugung.[4], [7]

Das Ergebnis der Generierung liegt zunächst noch in einer numerischen Form vor. Um daraus einen lesbaren Text zu machen, wird folgende Dekodierung durchgeführt:

```
caption = processor.decode(output[0], skip_special_tokens=True)
```

Listing 7 — caption.py Ausschnitt 6

Hier wird die vom Modell erzeugte Zahlenfolge wieder in natürlichen Text umgewandelt. Mit `skip_special_tokens=True` werden interne Steuerzeichen entfernt, sodass nur der eigentliche Beschreibungstext übrig bleibt. Das Resultat ist eine zusammenhängende, natürlichsprachliche Caption, die die im Bild erkannten Inhalte beschreibt.

Unmittelbar nach der Generierung wird das Ergebnis in einer strukturierten Datenform abgelegt:

```
result = {
  "image_path": str(image_path),
  "caption": caption
}
```

Listing 8 — caption.py Ausschnitt 7

Hier wird ein Dictionary erstellt, das sowohl den ursprünglichen Bildpfad als auch die erzeugte Caption enthält. Die Speicherung des Bildpfads ist wichtig für die Nachvollziehbarkeit, da so jederzeit eindeutig erkennbar bleibt, welche Beschreibung zu welchem Bild gehört. Diese zusätzliche Kontextinformation stellt sicher, dass auch bei späteren Auswertungen oder erneuten Durchläufen der Pipeline die Zuordnung erhalten bleibt.

Die eigentliche Speicherung erfolgt anschließend im JSON-Format:

```
output_path.parent.mkdir(parents=True, exist_ok=True)
output_path.write_text(
  json.dumps(result, ensure_ascii=False, indent=2),
  encoding="utf-8"
)
```

Listing 9 — caption.py Ausschnitt 8

Zunächst wird mit `mkdir(parents=True, exist_ok=True)` sichergestellt, dass der Zielordner existiert. Falls er noch nicht vorhanden ist, wird er automatisch erstellt. Dadurch wird verhindert, dass der Speichervorgang aufgrund fehlender Verzeichnisse fehlschlägt.

Mit `json.dumps(...)` wird das Dictionary in eine JSON-Struktur umgewandelt. Der Parameter `ensure_ascii=False` erlaubt die korrekte Speicherung von Umlauten oder Sonderzeichen. Durch `indent=2` wird die Datei formatiert abgespeichert, sodass sie gut lesbar bleibt. Schließlich wird die JSON-Datei mit UTF-8-Kodierung geschrieben, was eine stabile plattformübergreifende Weiterverarbeitung ermöglicht.

Die Generierung und Speicherung der Captions ist damit klar strukturiert und technisch sauber voneinander getrennt. Zunächst wird die Beschreibung automatisch durch das vortrainierte BLIP-Modell erzeugt, anschließend in lesbaren Text umgewandelt und schließlich gemeinsam mit dem Bildpfad in einer JSON-Datei gespeichert. Durch die Verwendung eines standardisierten Formats, konsistenter Pfadverwaltung und klarer Dateistruktur entsteht eine stabile Schnittstelle zwischen der Bildanalyse und den nachfolgenden Pipeline-Phasen. Damit ist die gespeicherte Caption mehr als ein Zwischenergebnis und bildet die Grundlage für die weiteren musikalischen Ableitungen.

# 5 Ableitung musikalischer Eigenschaften aus Text

## 5.1 Motivation für hybride Umsetzung der Zuordnung

Die Umsetzung der Ableitung musikalischer Eigenschaften in map_caption.py folgt bewusst einem hybriden Ansatz. Ziel dieses Schrittes ist es, die zuvor erzeugte Bildbeschreibung, also einen frei formulierten, natürlichen Text, in strukturierte musikalische Parameter zu überführen. Diese Transformation stellt eine Schnittstelle innerhalb der Pipeline dar, da hier der Übergang von sprachlicher Interpretation zu musikalischer Steuerung erfolgt.

Eine rein regelbasierte Lösung wäre an dieser Stelle nur eingeschränkt praktikabel gewesen. Bildbeschreibungen können sehr unterschiedlich formuliert sein. Selbst wenn zwei Bilder eine ähnliche Szene zeigen, können die generierten Captions sprachlich stark variieren. Unterschiedliche Wortwahl, Satzstrukturen oder semantische Nuancen würden bei einer ausschließlich schlüsselwortbasierten Verarbeitung schnell zu Problemen führen. Um alle möglichen Formulierungen durch feste Regeln abzudecken, wäre ein sehr umfangreiches und komplexes Regelsystem notwendig gewesen. Zudem bestünde die Gefahr, dass semantisch ähnliche Aussagen zu unterschiedlichen Ergebnissen führen, nur weil bestimmte Begriffe nicht exakt übereinstimmen.

Aus diesem Grund wird die semantische Einordnung der Caption, insbesondere die Ableitung von Stimmung und Genre, durch ein vortrainiertes Sprachmodell unterstützt. Diese Entscheidung ermöglicht es, flexibel mit unterschiedlich formulierten Texten umzugehen. Das Modell übernimmt die Aufgabe, die Bedeutung der Beschreibung zu interpretieren und sie in vorab definierte Kategorien einzuordnen. Dadurch wird vermieden, dass sprachliche Vielfalt zu instabilen oder inkonsistenten Ergebnissen führt.[9]

Gleichzeitig wurde jedoch bewusst darauf verzichtet, die gesamte Ableitung musikalischer Eigenschaften vollständig durch lernbasierte Verfahren zu realisieren. Nach der semantischen Einordnung erfolgt die konkrete Festlegung musikalischer Parameter, etwa Tempo oder Instrumentierung, regelbasiert. Dieser zweite Schritt ist deterministisch gestaltet und folgt klar

definierten Zuordnungen. Dadurch bleibt transparent, welche Annahmen über den Zusammenhang zwischen Stimmung, Genre und musikalischer Ausgestaltung getroffen werden.

Diese hybride Struktur vereint die Vorteile beider Ansätze. Die KI-gestützte Textinterpretation sorgt für Flexibilität und Robustheit gegenüber sprachlichen Variationen. Die regelbasierte Parametrisierung gewährleistet hingegen Nachvollziehbarkeit und Kontrolle. Identische oder sehr ähnliche Beschreibungen führen zu konsistenten musikalischen Einstellungen, was insbesondere für spätere Analysen wichtig ist. Unterschiede in den erzeugten Musikstücken lassen sich dadurch klarer auf die Musikgenerierung selbst zurückführen und nicht auf zufällige Schwankungen in der Parameterableitung.

Insgesamt wurde map_caption.py daher bewusst als vermittelnde Ebene zwischen textueller Bildbeschreibung und musikalischer Generierung konzipiert. Die Kombination aus lernbasierter Einordnung und regelbasierter Ausgestaltung schafft ein ausgewogenes Verhältnis zwischen Flexibilität, Transparenz und Kontrolle. Dadurch bleibt das System sowohl technisch robust als auch wissenschaftlich gut nachvollziehbar.

## 5.2 Struktur des Mapping-Skripts

Im Skript `map_caption.py` wird die zuvor erzeugte Caption eingelesen und in konkrete musikalische Eigenschaften übersetzt. Der Fokus liegt dabei darauf, aus einem frei formulierten Text eine kleine Menge strukturierter Werte abzuleiten, die später als „Steuerparameter“ für die Musikgenerierung dienen. Technisch besteht das Skript aus drei Hauptteilen: Einlesen der Eingabedatei und Extraktion der Caption, KI-gestützte Einordnung des Textes in Mood und Genre sowie anschließend die regelbasierte Ableitung von Tempo und Instrumentierung.

Zu Beginn stellt das Skript sicher, dass es korrekt aufgerufen wurde und die benötigten Kommandozeilenparameter vorhanden sind. Erwartet werden ein Pfad zur Eingabe-JSON und ein Pfad für die Ausgabedatei. Wenn diese fehlen, wird das Programm mit einer kurzen Usage-Meldung beendet, ähnlich wie am Anfang von `caption.py`.

Damit ist die Dateiverarbeitung von Anfang an klar definiert: `in_path` zeigt auf die Datei, aus der gelesen wird, und `out_path` definiert, wohin später geschrieben wird. Anschließend wird die Eingabedatei geladen und daraus gezielt die Caption extrahiert. Da in JSON-Dateien Werte auch fehlen können, ist der Zugriff defensiv umgesetzt: Wenn kein Caption-Feld vorhanden

ist, wird stattdessen ein leerer String verwendet. Durch `.strip()` werden zusätzlich führende und nachfolgende Leerzeichen entfernt, damit spätere Verarbeitungsschritte einen „sauberen“ Text erhalten:

```
data = json.loads(in_path.read_text(encoding="utf-8"))
caption = (data.get("caption") or "").strip()
```

Listing 10 — map_caption.py Ausschnitt 1

Der folgende Schritt setzt die Hauptidee des Skripts um: Der Text wird nicht regelbasiert analysiert, sondern von einem vortrainierten Sprachmodell in passende Kategorien eingeordnet. Dafür wird eine Hugging-Face-Pipeline für „Zero-Shot Classification„ initialisiert, die auf dem BART-Modell facebook/BART-large-mnli basiert [10]. Zero-Shot bedeutet hier vereinfacht: Das Modell kann den Text in vorgegebene Labels einsortieren, ohne dass es dafür extra auf genau diese Labels trainiert werden musste [9]. Diese Pipeline steht über die Transformers-Bibliothek zur Verfügung [5].

```
clf = pipeline("zero-shot-classification", model="facebook/bart-large-mnli")
```

Listing 11 — map_caption.py Ausschnitt 2

Wichtig ist dabei: Das Skript definiert selbst, welche Kategorien überhaupt erlaubt sind. Für die Stimmung werden feste Labels wie `calm`, `happy` oder `dark` vorgegeben. Für das Genre gibt es ebenfalls eine begrenzte Liste wie `ambient`, `cinematic` oder `rock`:

```
MOODS = ["calm", "happy", "sad", "dark", "energetic", "dramatic", "neutral"]
GENRES = ["ambient", "cinematic", "electronic", "pop", "rock", "jazz", "classical", "lofi", "folk"]
```

Listing 12 — map_caption.py Ausschnitt 3

Anschließend wird die Caption zweimal bewertet: einmal gegen die Mood-Liste und einmal gegen die Genre-Liste. Mit `multi_label=False` wird festgelegt, dass das Modell jeweils genau eine Kategorie auswählen soll. Das Ergebnis enthält eine sortierte Liste der Labels, wobei das wahrscheinlichste Label an erster Stelle steht:

```python
mood_res = clf(caption, MOODS, multi_label=False)
genre_res = clf(caption, GENRES, multi_label=False)

mood = mood_res["labels"][0]
genre = genre_res["labels"][0]
```

Listing 13 — map_caption.py Ausschnitt 4

Damit ist aus einem frei formulierten Text bereits eine erste strukturierte Interpretation entstanden: mood und genre sind nun zwei konkrete Werte aus einem festen Vokabular. Genau dieser Schritt macht den Ansatz robust, weil das Modell auch dann noch sinnvolle Zuordnungen treffen kann, wenn die Caption andere Wörter verwendet als man selbst in Regeln erwartet hätte.

Auf Basis dieser beiden Werte folgt anschließend die bewusst einfach gehaltene, regelbasierte Übersetzung in konkrete Musikparameter. Das Skript enthält dafür zwei Mapping-Tabellen: Eine ordnet jeder Stimmung ein Zieltempo zu, die andere ordnet jedem Genre eine typische Instrumentierung zu:

```python
TEMPO_BY_MOOD = {
  "calm": 75,
  "happy": 110,
  "sad": 70,
  "dark": 75,
  "energetic": 130,
  "dramatic": 85,
  "neutral": 100,
}

INSTR_BY_GENRE = {
  "ambient": ["pads", "soft synth"],
  "cinematic": ["strings", "pads", "low brass"],
  "electronic": ["drums", "synth", "bass"],
  "pop": ["drums", "bass", "keys"],
  "rock": ["drums", "electric guitar", "bass"],
  "jazz": ["piano", "upright bass", "drums"],
  "classical": ["strings", "piano"],
  "lofi": ["vinyl noise", "piano", "soft drums"],
  "folk": ["acoustic guitar", "light percussion"],
}
```

Listing 14 — map_caption.py Ausschnitt 5

Die konkrete Auswahl erfolgt dann über `.get(...)`, wodurch automatisch ein sinnvoller Standardwert genutzt wird, falls aus irgendeinem Grund ein unbekanntes Label auftaucht. Für Tempo ist das Default 100, für Instrumente `["pads"]`:

```
tempo_bpm = TEMPO_BY_MOOD.get(mood, 100)
instruments = INSTR_BY_GENRE.get(genre, ["pads"])
```

Listing 15 — map_caption.py Ausschnitt 6

Bis zu diesem Punkt hat `map_caption.py` damit seine Kernaufgabe erfüllt: Aus einer einzelnen Caption werden deterministische Steuerwerte abgeleitet. Damit die Entscheidungen nachvollziehbar bleiben, speichert das Skript zusätzlich die Scores des Zero-Shot-Modells. Diese Scores sind hilfreich, um später zu sehen, wie „sicher“ sich das Modell bei einer Einordnung war. Technisch werden dafür die Label- und Score-Listen zu Dictionaries zusammengeführt:

```
    data["music"] = {
    "mood": mood,
    "genre": genre,
    "tempo_bpm": tempo_bpm,
    "instruments": instruments,
    "debug": {
        "mood_scores": dict(zip(mood_res["labels"], mood_res["scores"])),
        "genre_scores": dict(zip(genre_res["labels"],
genre_res["scores"])),
    }
}
```

Listing 16 — map_caption.py Ausschnitt 7

Damit lässt sich map_caption.py insgesamt als „Übersetzer“ verstehen: Es nimmt einen natürlichen Beschreibungstext als Input, nutzt ein Sprachmodell, um diesen Text robust in feste Kategorien einzuordnen, und leitet daraus über klar definierte Regeln konkrete musikalische Steuerparameter ab. Der Ablauf ist dabei so gestaltet, dass die Textinterpretation flexibel bleibt, die eigentlichen Musikparameter aber kontrolliert und nachvollziehbar aus festen Zuordnungen entstehen.

## 5.3 Ausgabe und Weiterverarbeitung

Im Anschluss an die eigentliche Zuordnung von Mood, Genre, Tempo und Instrumenten übernimmt map_caption.py die Aufgabe, die ermittelten Werte so zu speichern, dass sie

von den nachfolgenden Pipeline-Schritten direkt weiterverwendet werden können. Dabei ist wichtig, dass die Ausgabe nicht als isoliertes Ergebnis entsteht, sondern die vorhandenen Daten aus der vorherigen Phase erweitert. Das Skript liest daher zuerst die bestehende JSON-Datei ein und ergänzt diese Struktur anschließend um ein neues Feld music. Genau dieses Vorgehen sorgt dafür, dass alle relevanten Informationen in einer einzigen, konsistenten Datei zusammenbleiben und später nicht „zusammengesucht“ werden müssen.

Wie die Ausgabe konkret aussieht, entspricht genau der Idee, dass die ursprüngliche Caption-Datei erweitert wird. Das folgende Beispiel zeigt eine typische JSON-Struktur nach dem Durchlauf von map_caption.py:

```
    {
  "image_path": "C:\\Projektpfad\\data\\images\\test.jpg",
  "caption": "a man in a yellow jersey holding a basketball ball",
  "music": {
    "mood": "dramatic",
    "genre": "lofi",
    "tempo_bpm": 85,
    "instruments": [
      "vinyl noise",
      "piano",
      "soft drums"
    ],
    "debug": {
      "mood_scores": {
        "dramatic": 0.2951374650001526,
        "energetic": 0.28246861696243286,
        "happy": 0.11736650764942169,
        "neutral": 0.10613572597503662,
        "dark": 0.086952805519104,
        "calm": 0.06940441578626633,
        "sad": 0.042534422129392624
      },
      "genre_scores": {
        "lofi": 0.24702782928943634,
        "cinematic": 0.24446122348308563,
        "folk": 0.1806531846523285,
        "ambient": 0.11045654863119125,
        "pop": 0.1032664030790329,
        "classical": 0.051368117332458496,
        "jazz": 0.02606263756752014,
        "rock": 0.019389161840081215,
        "electronic": 0.017314892262220383
      }
    }
  }
}
```

Listing 17 — beispiel Ausgabe von map_caption.py

Man erkennt hier gut das Prinzip: `image_path` und `caption` stammen aus dem vorherigen Schritt, und music wird durch `map_caption.py` ergänzt. Dadurch entsteht ein durchgängiges Datenobjekt, das sowohl für die Musikgenerierung als auch für spätere Analyse- oder Debug-Zwecke alle relevanten Informationen in einem einzigen, gut lesbaren Format enthält.

# 6 Generierung von Musik aus den abgeleiteten Eigenschaften

## 6.1 Auswahl des MusicGen-Modells

Die Auswahl des MusicGen-Modells ist ein wichtiger Entscheidungspunkt innerhalb der Musikgenerierungsphase der Pipeline. In diesem Schritt wird festgelegt, welches generative Modell für die Erzeugung von Musik aus den zuvor abgeleiteten textuellen und musikalischen Eigenschaften verwendet wird. Da die Musikgenerierung maßgeblichen Einfluss auf die Qualität der Gesamtergebnisse hat, musste ein Modell gewählt werden, das sowohl klanglich überzeugende Resultate liefert als auch praktikabel in die bestehende Systemarchitektur integrierbar ist. Die Entscheidung wurde dabei nicht ausschließlich anhand der maximal erreichbaren Audioqualität getroffen, sondern unter Berücksichtigung eines ausgewogenen Verhältnisses zwischen Qualität, Rechenaufwand und Integrationsaufwand.

MusicGen ist ein generatives Modell aus dem Bereich der KI, das speziell für die Erzeugung von Musik aus textuellen Beschreibungen entwickelt wurde. Das Modell ist darauf ausgelegt, aus natürlichsprachlichen Eingaben Audiosignale zu erzeugen, die bestimmte musikalische Eigenschaften widerspiegeln. Diese Eigenschaften können sowohl stilistische Aspekte wie Genre oder Stimmung als auch strukturelle Merkmale wie Tempo oder Instrumentierung umfassen. Durch diese Fähigkeiten eignet sich MusicGen besonders gut für den Einsatz in einer Pipeline, die auf textuellen Steuerinformationen basiert.[1], [11]

Ein wesentliches Auswahlkriterium war die verfügbare Modellgröße. MusicGen ist in mehreren Varianten erhältlich, die sich in ihrer Parameteranzahl und damit in ihrer Leistungsfähigkeit unterscheiden. Größere Modelle sind in der Lage, komplexere musikalische Strukturen zu erzeugen und weisen in der Regel eine höhere klangliche Qualität auf. Gleichzeitig erfordern sie jedoch deutlich mehr Rechenleistung und Speicherressourcen. Da das System nicht auf spezialisierten Hochleistungsrechnern ausgeführt werden soll, sondern auch auf handelsüblicher Hardware lauffähig sein muss, wurde bewusst eine kleinere Modellvariante gewählt. [1], [11]

Die gewählte Modellgröße stellt einen Kompromiss zwischen Audioqualität und Performance dar. Während größere Modelle potenziell detailliertere und abwechslungsreichere Musik erzeugen können, bietet die kleinere Variante eine deutlich kürzere Generierungszeit und einen geringeren Ressourcenverbrauch. Diese Eigenschaften sind insbesondere im Kontext einer interaktiven Anwendung relevant, bei der der Benutzer zeitnah Ergebnisse erwartet. Die Entscheidung für eine kompaktere Modellvariante unterstützt somit die Zielsetzung, ein praktikables und stabil laufendes System zu entwickeln.

Ein weiterer wichtiger Aspekt der Modellwahl betrifft die Integrationsfähigkeit. MusicGen wird als vortrainiertes Open-Source-Modell bereitgestellt und lässt sich über etablierte Programmierschnittstellen in Python einbinden.

Neben der reinen Audioqualität spielte auch die Stabilität der Modellausgaben eine Rolle bei der Auswahl. Generative Modelle können bei identischen Eingaben unterschiedliche Ergebnisse liefern, was grundsätzlich ein gewünschtes Merkmal kreativer Systeme ist. Gleichzeitig ist es jedoch wichtig, dass die erzeugten Audiosignale eine gewisse Konsistenz aufweisen, um einen sinnvollen Vergleich zwischen mehreren Versionen zu ermöglichen. Die gewählte MusicGen Variante zeigte in den Tests ein ausgewogenes Verhalten, bei dem Variabilität vorhanden ist, ohne dass die Ergebnisse völlig unvorhersehbar werden.

Insgesamt basiert die Auswahl des MusicGen Modells auf einer bewussten Abwägung zwischen technischer Machbarkeit, Ergebnisqualität und Systemperformance. Durch die Wahl einer geeigneten Modellgröße und die Nutzung eines offen verfügbaren Modells entsteht eine stabile Grundlage für die nachfolgende Musikgenerierung, die sowohl den praktischen Anforderungen der Anwendung als auch den wissenschaftlichen Zielen der Arbeit gerecht wird.

## 6.2 Erstellung der Musik

Im Skript musicgen.py wird der in der vorherigen Phase erzeugte strukturierte Datensatz, bestehend aus Caption sowie abgeleiteten musikalischen Parametern, in ein konkretes Audiosignal überführt. Dieser Schritt bildet den Übergang von symbolischer bzw. textueller Repräsentation zu tatsächlich hörbarer Musik. Der Ablauf lässt sich in drei Phasen gliedern: Einlesen der JSON-Daten, Konstruktion eines Steuertexts und Generierung sowie Speicherung der Audioausgabe.

Zu Beginn wird geprüft, ob das Skript mit den erforderlichen Parametern gestartet wurde. Erwartet werden der Pfad zur gemappten JSON-Datei sowie ein Basispfad für die spätere WAV-Ausgabe:

```
if len(sys.argv) < 3:
  print("Usage: python musicgen.py <input_mapped_json> <output_wav_base>")
  sys.exit(1)

in_path = Path(sys.argv[1])
out_wav_base = Path(sys.argv[2])
```

Listing 18 — musicgen.py Ausschnitt 1

Durch diese Prüfung wird sichergestellt, dass sowohl die Eingabedaten als auch das gewünschte Ausgabeverzeichnis eindeutig definiert sind. Anschließend wird die JSON-Datei eingelesen und in ein Python-Dictionary überführt:

```
data = json.loads(in_path.read_text(encoding="utf-8"))
caption = (data.get("caption") or "").strip()

music = data.get("music") or {}
mood = music.get("mood", "neutral")
genre = music.get("genre", "ambient")
tempo = music.get("tempo_bpm", 100)
instruments = music.get("instruments", ["pads"])
```

Listing 19 — musicgen.py Ausschnitt 2

Hier wird deutlich, dass musicgen.py direkt auf die Struktur der vorherigen Phase aufbaut. Aus der JSON werden die relevanten Parameter extrahiert: die textuelle Bildbeschreibung (caption) sowie die musikalischen Steuergrößen (mood, genre, tempo_bpm, instruments). Gleichzeitig werden Default-Werte definiert, falls einzelne Felder fehlen sollten. Dadurch bleibt der Prozess robust gegenüber unvollständigen Eingabedaten.

Im nächsten Schritt wird aus diesen Einzelinformationen ein zusammenhängender Steuertext erzeugt. Dieser Text fungiert als Eingabe für das Musikmodell und beschreibt in natürlicher Sprache, welche Art von Musik generiert werden soll:

```
inst_text = ", ".join(instruments) if isinstance(instruments, list) else
str(instruments)

control_text = (
  f"{genre} music, {mood} mood, tempo {tempo} bpm, instruments:
{inst_text}. "
  f"Scene: {caption}."
)
```

Listing 20 — musicgen.py Ausschnitt 3

Der Control-Text kombiniert Genre, Stimmung, Tempo und Instrumente mit der ursprünglichen Bildbeschreibung. Dadurch entsteht eine semantisch reichhaltige Beschreibung, die sowohl stilistische als auch inhaltliche Aspekte enthält. Das Modell erhält somit nicht nur abstrakte Parameter wie „85 bpm“, sondern zusätzlich eine szenische Einbettung über die Caption. Diese Kombination erhöht die Wahrscheinlichkeit, dass die generierte Musik thematisch mit dem Bildinhalt korrespondiert.

Anschließend wird das vortrainierte MusicGen-Modell geladen:

```
synth = pipeline("text-to-audio", model="facebook/musicgen-small")
```

Listing 21 — musicgen.py Ausschnitt 4

Es handelt sich hierbei um ein Text-zu-Audio-Modell, das aus einem gegebenen Textprompt ein Audiosignal erzeugt. Das Modell wird einmal initialisiert und anschließend mehrfach verwendet, um mehrere Versionen derselben inhaltlichen Vorgabe zu erzeugen.[1]

Vor der eigentlichen Generierung wird das Zielverzeichnis vorbereitet:

```
out_wav_base.parent.mkdir(parents=True, exist_ok=True)

base_stem = out_wav_base.stem
base_dir = out_wav_base.parent
```

Listing 22 — musicgen.py Ausschnitt 5

Hier wird sichergestellt, dass der Ausgabeordner existiert, und gleichzeitig wird der Basisname der Datei bestimmt. Dieser wird später um eine Versionsnummer ergänzt.

Die Generierung erfolgt innerhalb einer Schleife, um mehrere Varianten zu erzeugen:

```
N_VERSIONS = 3

for i in range(1, N_VERSIONS + 1):
  out = synth(
    control_text,
    forward_params={
        "do_sample": True,
        "max_new_tokens": 256
    }
)
```

Listing 23 — musicgen.py Ausschnitt 6

Mit `do_sample=True` wird das Modell nicht rein deterministisch betrieben, sondern darf innerhalb eines Wahrscheinlichkeitsraums variieren. Dadurch entstehen bei identischem Control-Text unterschiedliche, aber stilistisch passende Versionen. Diese Variation ist gewollt und bildet die Grundlage für spätere Vergleiche zwischen mehreren generierten Musikstücken.

Der Parameter `max_new_tokens=256` begrenzt die Länge der generierten Sequenz. Auch wenn hier intern keine klassischen „Wörter“, sondern Audioeinheiten erzeugt werden, wirkt dieser Wert faktisch als Steuergröße für die Dauer bzw. den Umfang des erzeugten Audios.

Das vom Modell zurückgegebene Objekt enthält unter anderem die Sampling-Rate und das eigentliche Audiosignal. Diese werden explizit extrahiert:

```
sr = int(out["sampling_rate"])
audio = np.asarray(out["audio"], dtype=np.float32)
```

Listing 24 — musicgen.py Ausschnitt 7

Die Sampling-Rate sr definiert, wie viele Samples pro Sekunde gespeichert werden. Typische Werte liegen beispielsweise bei 32.000 Hz oder 44.100 Hz. Das Feld audio enthält die eigentlichen Sample-Werte. Durch die Umwandlung in ein NumPy-Array mit dem Datentyp float32 wird sichergestellt, dass das Signal in einem konsistenten und speicherbaren Format vorliegt. [12]

Für jede Version wird anschließend ein eigener Dateiname erzeugt:

```
out_path = base_dir / f"{base_stem}_v{i}.wav"
```

Listing 25 — musicgen.py Ausschnitt 8

Dadurch entstehen Dateien wie `test1_v1.wav`, `test1_v2.wav` und `test1_v3.wav`. Diese klare Benennung ermöglicht eine eindeutige Zuordnung der Versionen und erleichtert spätere Vergleiche.

Die Speicherung erfolgt schließlich mit:

```
scipy.io.wavfile.write(out_path.as_posix(), sr, audio)
print("OK WAV:", out_path)
```

Listing 26 — musicgen.py Ausschnitt 9

Das generierte Audiosignal wird als WAV-Datei geschrieben. Das WAV-Format eignet sich besonders gut für Analyse- und Weiterverarbeitungsschritte, da es unkomprimiert ist und von nahezu allen Audio-Tools unterstützt wird.[13], [14]

Zusammenfassend wandelt musicgen.py die strukturierten JSON-Daten in ein konkretes Audiosignal um, indem ein semantisch angereicherter Steuertext an ein vortrainiertes Text-zu-Audio-Modell übergeben wird. Durch aktiviertes Sampling entstehen mehrere leicht variierende Versionen, die auf derselben inhaltlichen Grundlage basieren. Die generierten Audiodaten werden anschließend in einem standardisierten Format gespeichert, sodass sie für Analyse, Vergleich und weitere Pipeline-Schritte unmittelbar nutzbar sind.

# 7 Audioanalyse und Vergleich der generierten Musik

## 7.1 Ziel der Analyse

Das Ziel der Analyse besteht darin, die im Rahmen der Pipeline erzeugten Musikdaten systematisch zu untersuchen und vergleichbar zu machen. Da die Musikgenerierung auf generativen Verfahren aus dem Bereich der KI basiert, entstehen Ergebnisse, deren Eigenschaften nicht unmittelbar aus den Eingabedaten ableitbar sind. Die Analyse übernimmt in diesem Kontext die Aufgabe, die erzeugten Audiodaten über subjektive Höreindrücke hinaus quantitativ zu beschreiben und damit eine fundierte Grundlage für den Vergleich unterschiedlicher Musikversionen zu schaffen.

Ein wesentlicher Grund für die Durchführung einer Analyse liegt in der Mehrdeutigkeit der generierten Ergebnisse. Selbst bei identischen Eingaben in Form von Bild, Caption und Steuertext können deutlich unterschiedliche Musikversionen entstehen. Ohne eine systematische Analyse wäre es kaum möglich zu beurteilen, in welchen Aspekten sich diese Versionen unterscheiden und ob diese Unterschiede zufällig oder strukturell bedingt sind. Die Analyse ermöglicht es, Gemeinsamkeiten und Abweichungen sichtbar zu machen und damit das Verhalten des generativen Modells besser zu verstehen.

Darüber hinaus dient die Analyse der Objektivierung der Ergebnisse. Musik wird in der Regel subjektiv wahrgenommen, was Vergleiche zwischen verschiedenen Stücken erschwert. Durch die Extraktion messbarer Merkmale aus den Audiosignalen wird eine Ebene geschaffen, auf der Musik unabhängig von individuellen Höreindrücken beschrieben werden kann. Diese Merkmale liefern quantitative Informationen über Eigenschaften wie Dynamik, zeitliche Struktur oder spektrale Verteilung und erlauben damit einen systematischen Vergleich der erzeugten Musikversionen.

Ein weiterer Aspekt ist die Bewertung der Stabilität der Generierung. Da das System mehrere Musikversionen pro Bild erzeugt, stellt sich die Frage, inwieweit bestimmte musikalische Eigenschaften konstant bleiben oder variieren. Die Analyse ermöglicht es, diese Stabilität zu untersuchen, indem sie Unterschiede zwischen den Versionen messbar macht. Auf diese Weise

kann beurteilt werden, ob das System bei identischen Vorgaben reproduzierbare Tendenzen zeigt oder ob die Ergebnisse stark streuen.

Ein weiterer Beweggrund für die Analyse ist die Möglichkeit der automatisierten Auswertung. Da das System darauf ausgelegt ist, mehrere Bilder und mehrere Musikversionen zu verarbeiten, wäre eine rein manuelle Bewertung der Ergebnisse nicht praktikabel. Die Analyse schafft eine skalierbare Lösung, mit der große Mengen an Audiodaten effizient untersucht werden können. Dies ist insbesondere im Kontext experimenteller Untersuchungen von Bedeutung, bei denen zahlreiche Durchläufe der Pipeline durchgeführt werden.

Die Analyse ist zudem eng mit der Verwendung eines vortrainierten Open-Source-Modell verbunden. Da die internen Entscheidungsprozesse solcher Modelle nur begrenzt einsehbar sind, bietet die Auswertung der Ergebnisse eine indirekte Möglichkeit, das Modellverhalten zu untersuchen. Durch die Betrachtung der erzeugten Audiodaten lassen sich Rückschlüsse darauf ziehen, wie das Modell auf bestimmte Eingaben reagiert und welche musikalischen Muster es bevorzugt.

## 7.2 Extrahierte Audio-Features

Die quantitative Analyse der generierten Musikversionen erfolgt im Skript analyze.py durch die systematische Extraktion ausgewählter Audio-Features. Ziel dieses Schrittes ist es, das kontinuierliche Audiosignal in messbare Kennzahlen zu überführen, die unterschiedliche musikalische Eigenschaften repräsentieren. Dabei werden etablierte Funktionen der Bibliothek librosa verwendet, um sowohl zeitliche als auch spektrale Merkmale zu berechnen.

Zu Beginn der Verarbeitung wird jede WAV-Datei aus dem Ausgabeordner geladen. Dies geschieht innerhalb einer Schleife, die alle passenden Dateien durchläuft. Das Laden erfolgt mit:

```
y, sr = librosa.load(wav_path, sr=None, mono=True)
duration = float(librosa.get_duration(y=y, sr=sr))
```

Listing 27 — analyze.py Ausschnitt 1

Hierbei steht `y` für das Audiosignal selbst, während `sr` die Sampling-Rate darstellt. Durch `sr=None` wird die originale Sampling-Rate der Datei beibehalten, sodass keine automatische

Reskalierung stattfindet. Mit `mono=True` wird das Signal gegebenenfalls in ein einkanaliges Signal überführt. Dies vereinfacht die weitere Analyse, da alle Features auf einer konsistenten Signalstruktur basieren. Zusätzlich wird mit librosa.`get_duration` die Gesamtdauer des Signals berechnet, was eine grundlegende zeitliche Referenz darstellt.[15]

Ein weiteres wichtiges Feature ist der sogenannte RMS-Wert, der die mittlere quadratische Amplitude des Signals beschreibt. Er wird folgendermaßen berechnet:

```
rms = float(np.mean(librosa.feature.rms(y=y)))
```

Listing 28 — analyze.py Ausschnitt 2

librosa.`feature.rms` liefert zunächst eine zeitlich aufgelöste Folge von RMS-Werten. Durch `np.mean(...)` wird daraus ein einzelner Durchschnittswert über das gesamte Musikstück gebildet. Dieser Wert dient als Maß für die mittlere Energie bzw. wahrgenommene Lautheit. Höhere RMS-Werte deuten auf ein insgesamt energiereicheres Signal hin, während niedrigere Werte auf eine ruhigere oder dynamisch zurückhaltendere Musik schließen lassen.[15]

Zur Beschreibung der klanglichen Charakteristik wird der spektrale Schwerpunkt berechnet:

```
centroid = float(np.mean(librosa.feature.spectral_centroid(y=y, sr=sr)))
```

Listing 29 — analyze.py Ausschnitt 3

Der spektrale Schwerpunkt gibt an, in welchem Frequenzbereich der „Schwerpunkt“ der Energie liegt. Technisch wird dabei für jedes Zeitfenster der gewichtete Mittelwert der Frequenzen bestimmt. Ein höherer Durchschnittswert weist typischerweise auf einen helleren, brillanteren Klang hin, während niedrigere Werte eher dunklere oder wärmere Klangfarben repräsentieren. Durch die Mittelung über die Zeit entsteht ein globaler Kennwert, der sich gut für Vergleiche zwischen Versionen eignet.

Neben Energie und Klangfarbe spielt das Tempo eine wichtige Rolle. Das geschätzte Tempo wird über eine eigene Hilfsfunktion bestimmt:

```
def safe_tempo(y, sr):
try:
    tempo, _ = librosa.beat.beat_track(y=y, sr=sr)
    return float(np.squeeze(tempo))
except Exception:
    return np.nan
```

Listing 30 — analyze.py Ausschnitt 4

Und anschließend aufgerufen mit:

```
tempo_est = safe_tempo(y, sr)
```

Listing 31 — analyze.py Ausschnitt 5

librosa.`beat.beat_track` analysiert rhythmische Strukturen im Signal und schätzt daraus das Tempo in BPM. Da diese Berechnung nicht in jedem Fall erfolgreich sein muss, wird sie in eine try-except-Struktur eingebettet. Falls keine verlässliche Schätzung möglich ist, wird NaN zurückgegeben. Dadurch bleibt der Analyseprozess stabil, auch wenn einzelne Audiodateien keine klar erkennbaren rhythmischen Strukturen enthalten.[15]

Zur detaillierteren Beschreibung des Klangcharakters werden Mel-Frequency-Cepstral-Coefficients (MFCCs) berechnet:[16]

```
mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)
mfcc_mean = np.mean(mfcc, axis=1)
```

Listing 32 — analyze.py Ausschnitt 6

librosa.`feature`.MFCC berechnet hier 13 Koeffizienten pro Zeitfenster `(n_mfcc=13)`. Diese Koeffizienten repräsentieren eine komprimierte Darstellung der spektralen Struktur des Signals, die sich an der menschlichen Wahrnehmung orientiert.[15], [16]

Da die MFCCs zunächst zeitlich aufgelöst vorliegen, wird mit `np.mean(..., axis=1)` für jeden Koeffizienten ein Durchschnittswert über die gesamte Dauer gebildet. Das Ergebnis ist ein Vektor aus 13 Werten, der als kompakter akustischer „Fingerabdruck“ des Musikstücks interpretiert werden kann.[16]

Anschließend werden diese Werte einzeln in die Ergebnisstruktur übernommen:

```
for i, v in enumerate(mfcc_mean, start=1):
  row[f"mfcc{i}_mean"] = float(v)
```

Listing 33 — analyze.py Ausschnitt 7

Auf diese Weise erhält jeder MFCC-Koeffizient eine eigene Spalte (z. B. mfcc1_mean, mfcc2_mean, ...), wodurch spätere Vergleiche oder statistische Auswertungen erleichtert werden.[16]

Zusammenfassend extrahiert analyze.py für jede generierte Musikversion mehrere komplementäre Audio-Features: Die mittlere Energie (RMS), den spektralen Schwerpunkt als Maß für die Klanghelligkeit, eine Tempo-Schätzung sowie eine Reihe von MFCC-Koeffizienten als spektralen Fingerabdruck. Durch diese Kombination entsteht eine mehrdimensionale Beschreibung des Audiosignals, die sowohl zeitliche als auch klangliche Eigenschaften abdeckt. Die Berechnung erfolgt vollständig automatisiert auf Basis etablierter Methoden der digitalen Signalverarbeitung und bildet die quantitative Grundlage für alle weiteren Vergleichs- und Auswertungsschritte innerhalb der Pipeline.[16]

## 7.3 Vergleich mehrerer Versionen eines Bildes

Nachdem in analyze.py sämtliche Audio-Features für jede generierte WAV-Datei berechnet wurden, folgt der eigentliche Vergleich der Versionen. Ziel dieses Abschnitts ist es, nicht nur einzelne Kennzahlen zu speichern, sondern systematisch zu untersuchen, wie sich mehrere Musikversionen desselben Bildes voneinander unterscheiden. Dabei wird die zuvor erzeugte tabellarische Struktur genutzt, um Versionen automatisch zu gruppieren, Kennzahlen zu vergleichen und Unterschiede sichtbar zu machen.

Zunächst werden alle berechneten Feature-Werte, also RMS, spektraler Schwerpunkt, Tempo, MFCCs usw., in einer Liste gesammelt und anschließend in ein pandas-DataFrame überführt:[16]

```
df = pd.DataFrame(rows)
```

Listing 34 — analyze.py Ausschnitt 8

Jede Zeile dieses DataFrames entspricht genau einer generierten Musikversion. Um Versionen korrekt vergleichen zu können, muss jedoch zunächst erkannt werden, welche Dateien zu-

sammengehören. Da die Dateien nach dem Schema name_v1.wav, name_v2.wav usw. benannt sind, wird der Basisname vom Versionssuffix getrennt. Dies geschieht über eine kleine Hilfsfunktion:

```
def parse_version_from_filename(filename: str):
    stem = Path(filename).stem
    m = re.match(r"^(.*)_v(\d+)$", stem)
    if m:
        return m.group(1), int(m.group(2))
    return stem, None
```

Listing 35 — analyze.py Ausschnitt 9

Diese Funktion zerlegt beispielsweise test_v2.wav in:

- base_name = „test“
- version = 2

Anschließend wird diese Funktion auf jede Datei angewendet:

```
df[["base_name", "version"]] = df["file"].apply(
    lambda f: pd.Series(parse_version_from_filename(f))
)
```

Listing 36 — analyze.py Ausschnitt 10

Damit enthält das DataFrame nun explizite Spalten für den gemeinsamen Bildnamen sowie die jeweilige Versionsnummer. Erst durch diese Struktur wird ein systematischer Vergleich möglich.

Im nächsten Schritt erfolgt die Gruppierung aller Versionen nach ihrem gemeinsamen Basisnamen:

```
for base, g in df.groupby("base_name"):
    g = g.sort_values("version")
```

Listing 37 — analyze.py Ausschnitt 11

Hierbei repräsentiert `g` jeweils alle Versionen eines bestimmten Bildes. Durch `sort_values("version")` wird sichergestellt, dass die Versionen in der richtigen Reihenfolge betrachtet werden.

Ein Vergleich ist nur dann sinnvoll, wenn tatsächlich mehrere Versionen existieren. Daher wird zunächst geprüft:

```
if g["version"].notna().sum() < 2:
    continue
```

Listing 38 — analyze.py Ausschnitt 12

Nur wenn mindestens zwei Versionen vorhanden sind, wird der Vergleich durchgeführt.

Zunächst wird untersucht, wie stark sich die Versionen hinsichtlich ihrer mittleren Energie unterscheiden. Dazu werden Minimal- und Maximalwerte bestimmt:

```
rms_min, rms_max = g["rms"].min(), g["rms"].max()
```

Listing 39 — analyze.py Ausschnitt 13

Die Differenz rms_max - rms_min beschreibt die Spannweite der Lautheit zwischen den Versionen. Eine große Differenz deutet auf deutlich unterschiedliche Dynamiken hin, während eine kleine Differenz darauf schließen lässt, dass die Versionen energetisch sehr ähnlich sind.

Zusätzlich wird die Version mit dem höchsten RMS-Wert bestimmt:

```
loudest = g.loc[g["rms"].idxmax()]
```

Listing 40 — analyze.py Ausschnitt 14

Damit lässt sich eindeutig identifizieren, welche Version die größte mittlere Energie besitzt.

Analog dazu wird der spektrale Schwerpunkt betrachtet, der als Maß für die klangliche Helligkeit dient:

```
cen_min, cen_max = g["spectral_centroid"].min(), g["spectral_centroid"].max()
brightest = g.loc[g["spectral_centroid"].idxmax()]
```

Listing 41 — analyze.py Ausschnitt 15

Auch hier liefert die Differenz cen_max - cen_min eine quantitative Aussage darüber, wie stark sich die Klangcharakteristik zwischen den Versionen unterscheidet. Die Identifikation der Version mit dem höchsten Wert ermöglicht eine direkte Interpretation, etwa welche Variante klanglich am „hellsten“ wirkt.

Das geschätzte Tempo wird ebenfalls zwischen den Versionen verglichen:

```
tmp_min, tmp_max = g["tempo_est_bpm"].min(), g["tempo_est_bpm"].max()
fastest = g.loc[g["tempo_est_bpm"].idxmax()] if g["tempo_est_bpm"].notna().any() else None
```

Listing 42 — analyze.py Ausschnitt 16

Hier wird zunächst überprüft, ob überhaupt gültige Tempo-Schätzungen vorliegen. Falls dies der Fall ist, wird die schnellste Version bestimmt. Die Differenz tmp_max - tmp_min zeigt, ob sich die generierten Musikstücke rhythmisch deutlich unterscheiden oder weitgehend ähnliche Geschwindigkeiten aufweisen.

Die berechneten Vergleichswerte werden anschließend in einer neuen Struktur gesammelt:

```
comp_rows.append({
    "base_name": base,
    "versions_found": ",".join(str(int(v)) for v in g["version"].dropna().astype(int).tolist()),
    "rms_range": float(rms_max - rms_min),
    "centroid_range": float(cen_max - cen_min),
    "tempo_range": float(tmp_max - tmp_min) if np.isfinite(tmp_min) and np.isfinite(tmp_max) else np.nan,
    "loudest_version": int(loudest["version"]) if pd.notna(loudest["version"]) else None,
    "brightest_version": int(brightest["version"]) if pd.notna(brightest["version"]) else None,
    "fastest_version": int(fastest["version"]) if fastest is not None and pd.notna(fastest["version"]) else None,
})
```

Listing 43 — analyze.py Ausschnitt 17

Diese Struktur fasst für jedes Bild zusammen:

- Welche Versionen existieren
- Wie stark sie sich in Energie, Klangfarbe und Tempo unterscheiden
- Welche Version jeweils den höchsten Wert besitzt

Auf diese Weise entsteht eine verdichtete Vergleichsbasis, die nicht nur Einzelwerte betrachtet, sondern systematisch Unterschiede zwischen mehreren generierten Musikvarianten sichtbar macht.

## 7.4 Automatisch erzeugte Vergleichsberichte

Nachdem in analyze.py die Audio-Features extrahiert und die Versionen gruppiert sowie miteinander verglichen wurden, erfolgt im letzten Schritt die strukturierte Speicherung der Ergebnisse. Dabei werden zwei unterschiedliche Ausgabeformate erzeugt: eine vollständige CSV-Datei mit allen Einzelwerten sowie ein zusätzlich generierter Markdown-Bericht, der die wichtigsten Vergleichsergebnisse lesbar zusammenfasst.

Zunächst wird das DataFrame `df`, das alle berechneten Kennzahlen pro Musikversion enthält, als CSV-Datei gespeichert:

```
RES_DIR.mkdir(parents=True, exist_ok=True)
df.to_csv(CSV_OUT, index=False)
print("Wrote:", CSV_OUT)
```

Listing 44 — analyze.py Ausschnitt 18

Hier wird sichergestellt, dass das Zielverzeichnis existiert. Anschließend wird die gesamte Tabelle ohne zusätzlichen Index exportiert. Die CSV-Datei enthält dabei alle zuvor berechneten Werte, beispielsweise:

```
file,name,sr,duration_s,rms,spectral_centroid,tempo_est_bpm,...
test1_v1.wav,test1_v1,32000,5.06,0.1510800868,1666.1555,85.22727,...
```

Listing 45 — Beispiel csv

Diese Datei dient als vollständige Datengrundlage. Sie enthält sowohl globale Merkmale wie RMS oder spektralen Schwerpunkt als auch die einzelnen MFCC-Mittelwerte. Dadurch eignet sie sich besonders für weiterführende statistische Analysen, Diagrammerstellungen oder externe Auswertungen in anderen Programmen.[16]

Während die CSV-Datei alle Details enthält, ist sie für eine schnelle inhaltliche Bewertung weniger geeignet. Aus diesem Grund wird zusätzlich ein kompakter Vergleichsbericht erzeugt.

Neben der vollständigen CSV-Datei wird in analyze.py ein Markdown-Dokument erzeugt, das die Versionen pro Bild übersichtlich gegenüberstellt. Zunächst wird eine leere Liste vorbereitet, in der die einzelnen Textzeilen gesammelt werden:

```
report_lines = ["# Version Comparison Report\n"]
```

Listing 46 — analyze.py Ausschnitt 19

Für jede Gruppe von Versionen wird anschließend ein eigener Abschnitt erzeugt:

```
for base, g in df.groupby("base_name"):
    g = g.sort_values("version")
```

Listing 47 — analyze.py Ausschnitt 20

Zunächst wird eine tabellarische Übersicht der wichtigsten Kennzahlen erstellt:

```
report_lines.append(f"## {base}\n")
report_lines.append("| file | version | rms (energy) | spectral_centroid (brightness) | tempo_est_bpm |\n")
report_lines.append("|---|---:|---:|---:|---:|\n")

for _, r in g.iterrows():
    report_lines.append(
        f"| {r['file']} | {int(r['version']) if pd.notna(r['version']) else ''} | "
        f"{r['rms']:.4f} | {r['spectral_centroid']:.1f} | "
        f"{'' if pd.isna(r['tempo_est_bpm']) else f'{r['tempo_est_bpm']:.1f}'} |\n"
    )
```

Listing 48 — analyze.py Ausschnitt 21

Hier wird eine Markdown-Tabelle erzeugt, die pro Version folgende Werte enthält:

- Dateiname
- Versionsnummer
- RMS-Wert (Energie)
- spektraler Schwerpunkt (Helligkeit)
- geschätztes Tempo

Das Ergebnis sieht beispielsweise so aus:

```
## test
| file | version | rms (energy) | spectral_centroid (brightness) | tempo_est_bpm |
|---|---:|---:|---:|---:|
| test1_v1.wav | 1 | 0.1511 | 1666.2 | 85.2 |
| test1_v2.wav | 2 | 0.1148 | 1020.6 | 85.2 |
| test1_v3.wav | 3 | 0.0986 | 898.7 | 85.2 |
```

Listing 49 — Beispiel Mark-Down

Im Anschluss werden die zuvor berechneten Vergleichskennzahlen ergänzt. Dazu gehören unter anderem die Version mit dem höchsten RMS-Wert, der höchste spektrale Schwerpunkt sowie das höchste Tempo:

```
report_lines.append("\n**Highlights:**\n")
report_lines.append(f"- Loudest (highest RMS): **v{int(loudest['version'])}**\n")
report_lines.append(f"- Brightest (highest centroid): **v{int(brightest['version'])}**\n")
if fastest is not None and pd.notna(fastest["version"]):
    report_lines.append(f"- Fastest (highest tempo estimate): **v{int(fastest['version'])}**\n")
```

Listing 50 — analyze.py Ausschnitt 22

Zusätzlich werden die Spannweiten der wichtigsten Kennzahlen angegeben:

```
report_lines.append(f"- RMS range (max-min): **{(rms_max - rms_min):.4f}**\n")
report_lines.append(f"- Centroid range (max-min): **{(cen_max - cen_min):.1f} Hz**\n")
if np.isfinite(tmp_min) and np.isfinite(tmp_max):
    report_lines.append(f"- Tempo range (max-min): **{(tmp_max - tmp_min):.1f} BPM**\n")
```

Listing 51 — analyze.py Ausschnitt 23

Diese Angaben erlauben eine schnelle Interpretation, wie stark sich die Versionen unterscheiden. Beispielsweise zeigt eine hohe Centroid-Range, dass die Klangcharakteristik deutlich variiert, während eine Tempo-Range von 0.0 BPM auf identische Geschwindigkeiten hinweist.

Abschließend wird der gesamte Bericht als Markdown-Datei gespeichert:

```
report_md.write_text("".join(report_lines), encoding="utf-8")
```

Listing 52 — analyze.py Ausschnitt 24

Das Ergebnis ist ein lesbarer Bericht mit klar strukturierten Abschnitten und Tabellen, der sich beispielsweise direkt in GitHub, VS Code oder anderen Markdown-Viewern anzeigen lässt.

# 8 Orchestrierung der Pipeline mit C#

## 8.1 Aufruf und Steuerung der Python-Skripte

Die Steuerung der Python-Pipeline erfolgt in Program.cs über einen Orchestrierungsablauf, der die einzelnen Skripte als externe Prozesse startet, ihre Ausgaben einsammelt und Fehler konsequent behandelt. Dadurch bleibt die Python-Seite modular, während C# die Reihenfolge, die Dateipfade und die „Kontrolllogik“ übernimmt. Der Einstiegspunkt ist eine klassische Main()-Methode, in der zunächst die für die Ausführung relevanten Verzeichnisse und Skriptpfade aufgelöst werden:

```
var projectDir = Directory.GetCurrentDirectory();
var repoRoot = Path.GetFullPath(Path.Combine(projectDir, "..", ".."));

var imagesDir  = Path.Combine(repoRoot, "data", "images");
var resultsDir = Path.Combine(repoRoot, "data", "results");
var outputsDir = Path.Combine(repoRoot, "data", "outputs");

Directory.CreateDirectory(resultsDir);
Directory.CreateDirectory(outputsDir);

var captionScript = Path.Combine(repoRoot, "python", "caption.py");
var mapScript     = Path.Combine(repoRoot, "python", "map_caption.py");
var musicScript   = Path.Combine(repoRoot, "python", "musicgen.py");
var analyzeScript = Path.Combine(repoRoot, "python", "analyze.py");
```

Listing 53 — Program.cs Ausschnitt 1

Diese Pfadlogik sorgt dafür, dass die Pipeline unabhängig vom konkreten Startverzeichnis zuverlässig auf die Repository-Struktur zugreifen kann. Direkt im Anschluss werden die erwarteten Skripte und Eingabeordner geprüft, um Fehler möglichst früh und eindeutig abzufangen. Dadurch wird verhindert, dass ein Prozess gestartet wird, der anschließend wegen fehlender Dateien scheitert:

```
if (!File.Exists(captionScript))
    throw new FileNotFoundException("caption.py nicht gefunden",
captionScript);
if (!File.Exists(mapScript))
    throw new FileNotFoundException("map_caption.py nicht gefunden",
mapScript);
if (!File.Exists(musicScript))
    throw new FileNotFoundException("musicgen.py nicht gefunden",
musicScript);
if (!File.Exists(analyzeScript))
    throw new FileNotFoundException("analyze.py nicht gefunden",
analyzeScript);
if (!Directory.Exists(imagesDir))
    throw new DirectoryNotFoundException($"images-Ordner nicht gefunden:
{imagesDir}");
```

Listing 54 — Program.cs Ausschnitt 2

Ein praktischer Aspekt der Orchestrierung ist die Auswahl des Python-Interpreters. Anstatt pauschal python aufzurufen, wird bevorzugt ein lokales Virtual Environment verwendet, falls es im Repository vorhanden ist. Das stellt sicher, dass die installierten Abhängigkeiten konsistent sind und nicht von einer globalen Python-Installation abhängen:

```
var venvPython = Path.Combine(repoRoot, ".venv", "Scripts", "python.exe");
var pythonExe = File.Exists(venvPython) ? venvPython : "python";
```

Listing 55 — Program.cs Ausschnitt 3

Für die Steuerung, welche Eingaben verarbeitet werden, werden die verfügbaren Bilddateien gesammelt und auf typische Endungen eingeschränkt. Das ist ebenfalls Teil der „Kontrolle“ der Pipeline, weil dadurch nur valide Eingaben in die nachfolgenden Schritte gelangen:

```
var allImages = Directory.EnumerateFiles(imagesDir)
    .Where(f => f.EndsWith(".jpg", StringComparison.OrdinalIgnoreCase)
             || f.EndsWith(".jpeg", StringComparison.OrdinalIgnoreCase)
             || f.EndsWith(".png", StringComparison.OrdinalIgnoreCase)
             || f.EndsWith(".webp", StringComparison.OrdinalIgnoreCase))
    .ToList();
```

Listing 56 — Program.cs Ausschnitt 4

Anschließend erfolgt eine einfache Benutzerauswahl. Aus Sicht der Orchestrierung ist entscheidend, dass daraus eine Liste „selectedImages“ entsteht, die später die Schleife bestimmt,

in der die Python-Skripte sequenziell aufgerufen werden. Die eigentliche Pipeline-Ausführung ist dann sehr klar: pro Bild werden zuerst die Zielpfade für Zwischen- und Endartefakte definiert und anschließend die Skripte in fester Reihenfolge gestartet. Damit wird die Datenabhängigkeit explizit abgebildet, da jeder Schritt auf dem Output des vorherigen Schritts basiert:

```
foreach (var img in selectedImages)
{
    var name = Path.GetFileNameWithoutExtension(img);

    var captionJson = Path.Combine(resultsDir, $"{name}.json");
    var mappedJson  = Path.Combine(resultsDir, $"{name}_mapped.json");
    var outWav      = Path.Combine(outputsDir, $"{name}.wav");

    RunPython(pythonExe, captionScript, $"\"{img}\" \"{captionJson}\"");
    RunPython(pythonExe, mapScript,     $"\"{captionJson}\" \"{mappedJson}\"");
    RunPython(pythonExe, musicScript,   $"\"{mappedJson}\" \"{outWav}\"");
}
```

Listing 57 — Program.cs Ausschnitt 5

Wichtig ist hier die konsequente Übergabe über Kommandozeilenargumente. Dadurch bleiben die Python-Skripte unabhängig von `C#` und können auch separat getestet werden. Gleichzeitig wird in `C#` durch das konsequente `Quoting (\"...\")` sichergestellt, dass Pfade mit Leerzeichen korrekt verarbeitet werden, was unter Windows besonders relevant ist.

Nach der Erzeugung der WAV-Dateien wird am Ende eine Analyse über alle erzeugten Audios gestartet. Die Orchestrierung trennt damit bewusst „pro Bild“-Schritte (Caption/Mapping/Music) von einem abschließenden Gesamtschritt (Analyse), was dem Pipeline-Charakter entspricht:

```
RunPython(pythonExe, analyzeScript, "");
```

Listing 58 — Program.cs Ausschnitt 6

Der Kern der technischen Steuerung liegt in der Hilfsmethode `RunPython(...)`. Sie kapselt den Start eines Python-Prozesses, das Sammeln der Ausgaben sowie das Fehlerhandling.

Dadurch bleibt der restliche Code übersichtlich, und es gibt einen einheitlichen Mechanismus, um alle Skripte gleich zu starten:

```
static void RunPython(string pythonExe, string scriptPath, string args)
{
    var psi = new ProcessStartInfo
    {
        FileName = pythonExe,
        Arguments = string.IsNullOrWhiteSpace(args)
            ? $"\"{scriptPath}\""
            : $"\"{scriptPath}\" {args}",
        RedirectStandardOutput = true,
        RedirectStandardError = true,
        UseShellExecute = false,
        CreateNoWindow = true
    };

    using var p = Process.Start(psi);
    if (p == null) throw new Exception("Konnte Python-Prozess nicht starten.");

    var stdout = p.StandardOutput.ReadToEnd();
    var stderr = p.StandardError.ReadToEnd();
    p.WaitForExit();

    if (!string.IsNullOrWhiteSpace(stdout)) Console.WriteLine(stdout.Trim());
    if (!string.IsNullOrWhiteSpace(stderr)) Console.WriteLine(stderr.Trim());

    if (p.ExitCode != 0)
        throw new Exception($"Python-Skript fehlgeschlagen: {Path.GetFileName(scriptPath)} (ExitCode {p.ExitCode})");
}
```

Listing 59 — Program.cs Ausschnitt 7

Die Konfiguration `RedirectStandardOutput = true` und `RedirectStandardError = true` stellt sicher, dass sowohl normale Programmausgaben als auch Fehlermeldungen zuverlässig in `C#` ankommen und protokolliert werden können. `UseShellExecute = false` ist dabei zwingend nötig, damit die Umleitung der Streams überhaupt funktioniert. `CreateNoWindow = true` verhindert, dass für jeden Prozess ein separates Konsolenfenster geöffnet wird, wodurch die Steuerung für den Benutzer deutlich sauberer wirkt.

Für die Robustheit ist insbesondere das Exit-Code-Handling entscheidend. Sobald ein Python-Skript mit einem Fehler endet (ExitCode ungleich 0), wird in `C#` eine Exception ausgelöst.

Dadurch bricht die Pipeline kontrolliert ab, anstatt still weiterzulaufen und Folgefehler durch fehlende oder inkonsistente Zwischenergebnisse zu erzeugen. Zusammen mit dem umgebenden try/catch in Main() entsteht so eine klare Fehlerkommunikation:

```
try
{
    // Pipeline
    return 0;
}
catch (Exception ex)
{
    Console.WriteLine("\nFEHLER");
    Console.WriteLine(ex.ToString());
    return 1;
}
```

Listing 60 — Program.cs Ausschnitt 8

In Summe übernimmt Program.cs damit die Rolle eines Orchestrators: Es wählt einen konsistenten Python-Interpreter, bereitet Pfade und Ordner vor, startet die Python-Skripte in einer festen Reihenfolge, übergibt alle benötigten Parameter über die Kommandozeile, sammelt Standardausgabe und Fehlermeldungen ein und erzwingt über Exit-Codes ein zuverlässiges Abbruchverhalten im Fehlerfall. Dadurch wird die Pipeline steuerbar, reproduzierbar und gleichzeitig modular gehalten, weil die einzelnen Verarbeitungsschritte weiterhin in separaten Python-Skripten gekapselt bleiben.[2]

## 8.2 Automatisierte Ausführung der gesamten Pipeline

Neben der reinen Prozesssteuerung übernimmt die Program.cs die Verantwortung für den gesamten Lebenszyklus eines Durchlaufs. Die Automatisierung beginnt dabei nicht erst beim Skriptaufruf, sondern bereits bei der Vorbereitung der Arbeitsumgebung. Um eine fehlerfreie Ausführung zu garantieren, stellt die Pipeline sicher, dass alle benötigten Verzeichnisse für Zwischenergebnisse und Endprodukte existieren, bevor die erste Rechenoperation stattfindet. Dies verhindert Laufzeitfehler durch fehlende Dateipfade in den nachgelagerten Python-Skripten:

```
var resultsDir = Path.Combine(repoRoot, "data", "results");
var outputsDir = Path.Combine(repoRoot, "data", "outputs");

Directory.CreateDirectory(resultsDir);
Directory.CreateDirectory(outputsDir);
```

Listing 61 — Program.cs Ausschnitt 9

Ein wesentlicher Aspekt der Automatisierung ist die intelligente Filterung der Eingabedaten. Die Pipeline erkennt selbstständig valide Bilddateien im Quellverzeichnis. Durch den Einsatz von LINQ wird eine Liste von Pfaden erstellt, die den unterstützten Formaten entsprechen. Dies entlastet den Benutzer davon, Dateipfade manuell anzugeben oder sich um die Dateiendungen zu kümmern:

```
var allImages = Directory.EnumerateFiles(imagesDir)
  .Where(f => f.EndsWith(".jpg", StringComparison.OrdinalIgnoreCase) ||
f.EndsWith(".jpeg", StringComparison.OrdinalIgnoreCase) ||
f.EndsWith(".png", StringComparison.OrdinalIgnoreCase) ||
f.EndsWith(".webp", StringComparison.OrdinalIgnoreCase)) .ToList();
```

Listing 62 — Program.cs Ausschnitt 10

Obwohl das System vollautomatisch arbeiten kann, integriert es eine flexible Benutzerschnittstelle für die Selektion der Arbeitslast. Der Orchestrator bietet hierbei zwei Modi an: Die Verarbeitung des gesamten Datensatzes mittels des Schlüsselworts all oder eine gezielte Auswahl spezifischer Dateien über deren Namen. Die Automatisierung zeigt sich hier in der robusten Verarbeitung der Benutzereingabe, die Leerzeichen und Trennzeichen bereinigt, um die Auswahl mit dem tatsächlichen Dateisystem abzugleichen:

```
var wantedNames = input
  .Split(new[] { ',', ' ' }, StringSplitOptions.RemoveEmptyEntries)
  .Select(s => s.Trim())
  .Where(s => s.Length > 0)
  .ToHashSet(StringComparer.OrdinalIgnoreCase);

selectedImages = allImages
  .Where(path =>
wantedNames.Contains(Path.GetFileNameWithoutExtension(path)))
  .ToList();
```

Listing 63 — Program.cs Ausschnitt 11

Sobald die Auswahl feststeht, transformiert die Pipeline die abstrakte Liste von Bildern in eine konkrete Abfolge von Arbeitsschritten. Für jedes Bild wird automatisiert ein konsistentes Benennungsschema für die Artefakte abgeleitet. Diese Namenskonvention zieht sich durch alle Phasen, von der initialen Bildbeschreibung über die musikalische Parametrierung bis hin zur finalen Audio-Generierung. Dadurch entsteht eine lückenlose „Chain of Custody“ der Daten, die für die spätere Analyse essenziell ist:

```
foreach (var img in selectedImages)
{
var name = Path.GetFileNameWithoutExtension(img);
var captionJson = Path.Combine(resultsDir, $"{name}.json");
var mappedJson  = Path.Combine(resultsDir, $"{name}_mapped.json");
var outWav      = Path.Combine(outputsDir, $"{name}.wav");

// Sequenzieller Pipeline-Aufruf...
}
```

Listing 64 — Program.cs Ausschnitt 12

Den Abschluss der automatisierten Kette bildet der Übergang von der Einzelbildverarbeitung zur aggregierten Datenanalyse. Nachdem alle gewählten Bilder die Pipeline durchlaufen haben, stößt der Orchestrator ohne weiteres Zutun des Benutzers das Analyse-Skript an. Dieser Schritt schließt den Kreis der Automatisierung: Er sammelt die verteilten Einzelergebnisse ein und überführt sie in eine strukturierte Form, wodurch der gesamte Prozess von der rohen Bilddatei bis zum evaluierten Ergebnis in einem einzigen, ununterbrochenen Durchlauf abgeschlossen wird.

# 9 Entwicklung der grafischen Benutzeroberfläche

## 9.1 Zielsetzung der Benutzeroberfläche

Die Zielsetzung der Benutzeroberfläche besteht darin, den komplexen Ablauf der Bild zu Musik Pipeline in einer verständlichen und intuitiv bedienbaren Form zugänglich zu machen. Da das zugrunde liegende System aus mehreren Verarbeitungsschritten besteht und sowohl KI basierte Modelle als auch klassische Softwarekomponenten umfasst, wäre eine rein technische Nutzung über Kommandozeilen oder Konfigurationsdateien nur eingeschränkt praktikabel. Die Benutzeroberfläche dient daher als vermittelnde Ebene zwischen der technischen Implementierung und dem Anwender und ermöglicht eine komfortable Nutzung des Systems ohne detaillierte Kenntnisse der internen Abläufe.

Eines der Hauptziele der Benutzeroberfläche ist die Reduktion der Komplexität. Der Anwender soll in der Lage sein, ein Bild auszuwählen, den Verarbeitungsprozess zu starten und die erzeugten Ergebnisse zu betrachten, ohne sich mit einzelnen Skripten, Dateipfaden oder Parametern auseinandersetzen zu müssen. Die Oberfläche abstrahiert die technischen Details und stellt nur die für den Nutzer relevanten Interaktionsmöglichkeiten bereit. Dadurch wird das System auch für Personen zugänglich, die keine Programmier oder KI Vorkenntnisse besitzen.

Die Benutzeroberfläche verfolgt darüber hinaus das Ziel, den Ablauf der Pipeline transparent darzustellen. Obwohl die technischen Details im Hintergrund verborgen bleiben, soll der Nutzer jederzeit erkennen können, in welchem Zustand sich das System befindet. Rückmeldungen über laufende Verarbeitungsschritte, abgeschlossene Phasen oder auftretende Fehler tragen dazu bei, Vertrauen in den Ablauf zu schaffen und die Nachvollziehbarkeit zu erhöhen. Die Oberfläche fungiert damit nicht nur als Bedienwerkzeug, sondern auch als visuelle Repräsentation des Systemzustands.

Ein weiterer wichtiger Aspekt der Zielsetzung ist die Unterstützung der Analysephase. Die Benutzeroberfläche soll nicht nur die Musikgenerierung ermöglichen, sondern auch den Zugriff auf die erzeugten Analyseergebnisse erleichtern. Durch die Anzeige von Vergleichsberichten oder Analyseinformationen können die Ergebnisse unmittelbar im Kontext betrachtet

werden, ohne externe Werkzeuge nutzen zu müssen. Dies unterstützt die Zielsetzung der Arbeit, die Ergebnisse der KI basierten Musikgenerierung systematisch zu untersuchen und anschaulich darzustellen.

Zusammenfassend dient die Benutzeroberfläche als Schnittstelle zwischen Mensch und System. Sie verbindet die technische Komplexität der Pipeline mit einer klaren und verständlichen Interaktion und unterstützt damit sowohl die Nutzung als auch die Analyse der erzeugten Ergebnisse. Die Zielsetzung der Benutzeroberfläche ist es, den Zugang zur Bild-zu-Musik-Generierung zu vereinfachen und zugleich die Struktur und Funktionsweise des Systems transparent zu machen.

## 9.2 Auswahl von WPF und Projektintegration

Die Auswahl von WPF als Technologie für die Umsetzung der grafischen Benutzeroberfläche ergibt sich aus den Anforderungen an Struktur, Erweiterbarkeit und enge Integration in die bestehende `C#` Anwendung. Da die Benutzeroberfläche nicht als isoliertes Zusatzmodul, sondern als fester Bestandteil des Gesamtsystems konzipiert ist, musste eine Lösung gewählt werden, die eine saubere Verbindung zwischen Anwendungslogik und Darstellung erlaubt. WPF bietet hierfür eine geeignete Grundlage, da es speziell für die Entwicklung komplexer Desktop-Anwendungen innerhalb des `C#` Ökosystems ausgelegt ist.

Ein Vorteil von WPF liegt in der klaren Trennung zwischen Benutzeroberfläche und Programmlogik. Durch dieses Konzept können visuelle Elemente unabhängig von der zugrunde liegenden Logik gestaltet werden. Diese Trennung unterstützt eine übersichtliche Projektstruktur und erleichtert die Wartung des Codes. Gerade im Kontext einer Anwendung, die mehrere komplexe Verarbeitungsschritte steuert und den aktuellen Systemzustand darstellen muss, ist diese klare Struktur von besonderer Bedeutung.

Die Integration von WPF in das bestehende Projekt erfolgte nahtlos, da die Orchestrierungslogik bereits in `C#` umgesetzt ist. Die Benutzeroberfläche greift direkt auf diese Logik zu und fungiert als visuelle Steuerungsschicht. Aktionen des Benutzers wie das Auswählen eines Bildes oder das Starten der Pipeline lösen klar definierte Methoden innerhalb der Orchestrierung aus. Dadurch entsteht ein konsistenter Ablauf, bei dem Benutzerinteraktionen und interne Verarbeitungsschritte eng miteinander verknüpft sind.

Ein weiterer Aspekt der Entscheidung für WPF ist die Flexibilität bei der Gestaltung der Benutzeroberfläche. WPF ermöglicht die Umsetzung komplexer Layouts und dynamischer Inhalte, die sich während der Laufzeit verändern können. Dies ist insbesondere relevant für die Darstellung des Pipeline Zustands, von Fortschrittsinformationen oder von Analyseergebnissen. Die Oberfläche kann auf Änderungen im Systemzustand reagieren und dem Benutzer jederzeit aktuelles Feedback geben.

Insgesamt ermöglicht die Auswahl von WPF eine enge und saubere Integration der grafischen Benutzeroberfläche in das bestehende Projekt. Die Technologie unterstützt die strukturelle Trennung von Darstellung und Logik, bietet hohe Flexibilität bei der Gestaltung und fügt sich nahtlos in die `C#` basierte Orchestrierung ein. Dadurch entsteht eine Benutzeroberfläche, die nicht nur funktional, sondern auch gut wartbar und erweiterbar ist.[17]

## 9.3 Aufbau des UI-Layouts

Die grafische Benutzeroberfläche wurde mit der Windows Presentation Foundation (WPF) realisiert. Das Ziel des Layout-Designs war eine klare, vertikale Prozessführung, die den Benutzer durch die Pipeline leitet. Um eine strukturierte Anordnung der verschiedenen Steuerelemente zu erreichen, basiert das Hauptlayout auf einem Grid-System mit vier Zeilendefinitionen. Dies erlaubt es, die Interaktionselemente im oberen Bereich zu fixieren, während das Log-Fenster dynamisch den restlichen verfügbaren Platz einnimmt:[17]

```
<Grid Margin="12">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*"/>
        </Grid.RowDefinitions>
        ...
</Grid>
```

Listing 65 — MainWindow.xaml Ausschnitt 1

Wie in der folgenden Abbildung zu sehen ist, spiegelt die Oberfläche den Workflow wider: Von der Auswahl der Quelldaten über die Ausführung bis hin zur Evaluation der Ergebnisse.

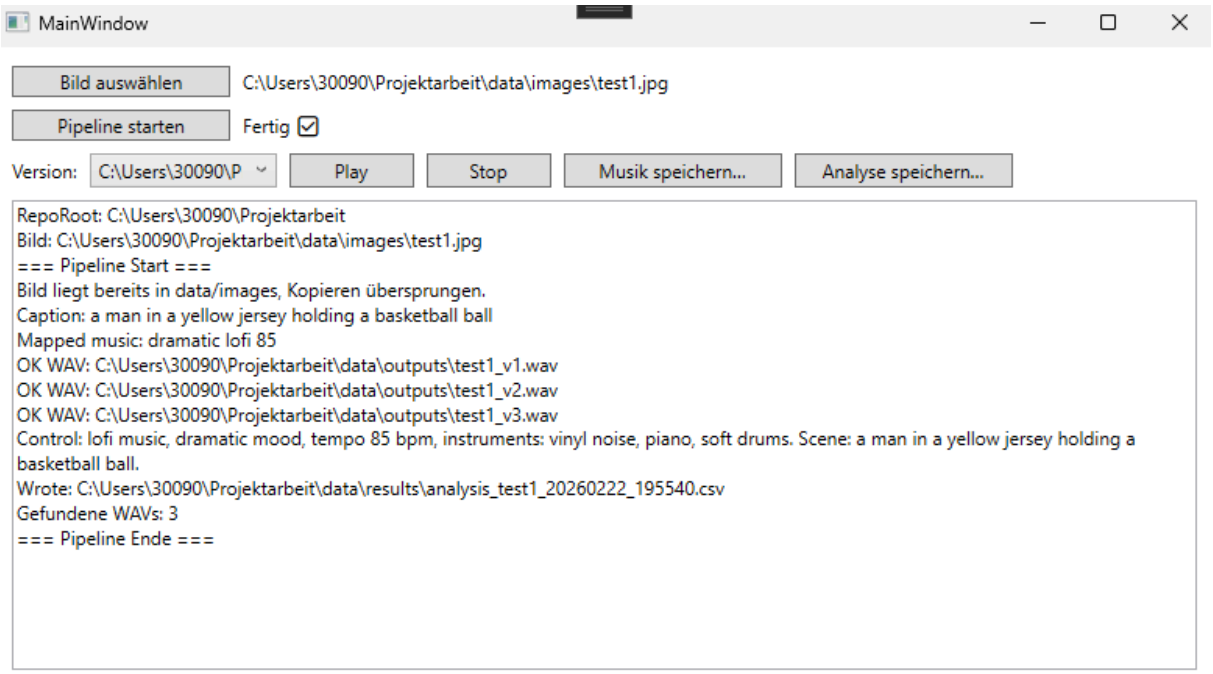


Abbildung 3 — Benutzeroberfläche der WPF-Anwendung

Im ersten Bereich der Benutzeroberfläche befindet sich die Bildauswahl. Hier wird ein StackPanel mit horizontaler Ausrichtung genutzt, um einen Button und ein TextBlock-Element nebeneinander zu platzieren. Letzteres ist über Data Binding mit der Eigenschaft SelectedImageText verknüpft, sodass der Benutzer sofort Feedback erhält, welche Datei für die Verarbeitung geladen wurde:

```
<StackPanel Orientation="Horizontal" Grid.Row="0" Margin="0,0,0,8">
            <Button Width="140" Margin="0,0,8,0" Content="Bild auswählen" Click="PickImage_Click"/>
            <TextBlock VerticalAlignment="Center" Text="{Binding SelectedImageText}" />
        </StackPanel>
```

Listing 66 — MainWindow.xaml Ausschnitt 2

Der funktionale Kern des Layouts ist die Sektion für die Audio-Steuerung. Da nach der Ausführung der Pipeline oft mehrere Versionen oder Ergebnisse vorliegen können, wurde eine ComboBox integriert. Diese dient als Dropdown-Menü zur Auswahl der generierten WAV-Dateien. Die daneben liegenden Schaltflächen für „Play“, „Stop“ und die Speicherfunktionen sind bewusst kompakt gruppiert, um eine intuitive Mediensteuerung innerhalb der Applikation zu ermöglichen:

```
<StackPanel Orientation="Horizontal" Grid.Row="2" Margin="0,0,0,8">
            <TextBlock VerticalAlignment="Center" Text="Version:"
Margin="0,0,8,0"/>
            <ComboBox Width="120" Margin="0,0,8,0" ItemsSource="{Binding
AudioFiles}" SelectedItem="{Binding SelectedAudioFile}"/>
            <Button Width="80" Margin="0,0,8,0" Content="Play"
Click="Play_Click"/>
            <Button Width="80" Margin="0,0,8,0" Content="Stop"
Click="Stop_Click"/>
            <Button Width="140" Margin="0,0,8,0" Content="Musik
speichern…" Click="SaveWav_Click"/>
            <Button Width="140" Content="Analyse speichern…"
Click="SaveAnalysis_Click"/>
        </StackPanel>
```

Listing 67 — MainWindow.xaml Ausschnitt 3

Um die Transparenz während des rechenintensiven Prozesses zu wahren, nimmt die Log-Anzeige den größten Teil der Fläche ein. Eine TextBox im unteren Bereich `(Grid.Row="3")` fungiert als Konsole. Durch die Eigenschaften VerticalScrollBarVisibility=„Auto“ und IsReadOnly=„true“ wird sichergestellt, dass der Benutzer den Fortschritt der Python-Skripte mitverfolgen kann, ohne die Protokollierung versehentlich zu manipulieren. Das Binding an LogText sorgt dafür, dass die Ausgaben der RunPython-Methode in Echtzeit im UI erscheinen:

```
<TextBox Grid.Row="3"
             Text="{Binding LogText}"
             IsReadOnly="True"
             TextWrapping="Wrap"
             VerticalScrollBarVisibility="Auto"
             HorizontalScrollBarVisibility="Auto"/>
```

Listing 68 — MainWindow.xaml Ausschnitt 4

Durch diese klare Segmentierung trennt das UI-Layout die administrativen Aufgaben (Pfade, Auswahl) von den operativen Aufgaben (Start, Playback) und den informativen Aufgaben (Log, Analyse).

## 9.4 Interaktion zwischen UI und Pipeline

Die Interaktion zwischen der grafischen Benutzeroberfläche und der KI-Pipeline wird in der Anwendung direkt in der Klasse MainWindow umgesetzt. Diese Klasse implementiert das

Interface „INotifyPropertyChanged“, wodurch Änderungen an Eigenschaften automatisch an das UI weitergegeben werden können.

```
public partial class MainWindow : Window, INotifyPropertyChanged
```

Listing 69 — MainWindow.xaml.cs Ausschnitt 1

Die Benachrichtigung erfolgt über die Methode OnPropertyChanged:

```
public event PropertyChangedEventHandler? PropertyChanged;
private void OnPropertyChanged([CallerMemberName] string? name = null)
    => PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
```

Listing 70 — MainWindow.xaml.cs Ausschnitt 2

Damit werden gebundene UI-Elemente automatisch aktualisiert, sobald sich eine Eigenschaft ändert.

**Datenbindung und Statusaktualisierung**

Der aktuelle Zustand der Pipeline wird über die Eigenschaft StatusText an das UI gebunden:

```
private string _statusText = "Bereit";
public string StatusText
{
    get => _statusText;
    set
    {
        _statusText = value;
        OnPropertyChanged();
    }
}
```

Listing 71 — MainWindow.xaml.cs Ausschnitt 3

Sobald der Status verändert wird, aktualisiert sich der entsprechende TextBlock in der Oberfläche automatisch.

Dasselbe Prinzip wird für das Log-Feld verwendet:

```
private string _logText = "";
public string LogText
{
    get => _logText;
    set
    {
        _logText = value;
        OnPropertyChanged();
    }
}
```

Listing 72 — MainWindow.xaml.cs Ausschnitt 4

Neue Einträge werden über AppendLog() ergänzt:

```
private void AppendLog(string text)
{
    Dispatcher.Invoke(() =>
    {
        LogText += (LogText.Length == 0 ? "" : "\n") + text;
    });
}
```

Listing 73 — MainWindow.xaml.cs Ausschnitt 5

Da Teile der Pipeline in Hintergrund-Threads laufen, ist hier Dispatcher.Invoke() notwendig, um UI-Updates thread-sicher auszuführen.

**Start der Pipeline**

Die Pipeline wird über den Button-Handler RunPipeline_Click gestartet:

```
private async void RunPipeline_Click(object sender, RoutedEventArgs e)
```

Listing 74 — MainWindow.xaml.cs Ausschnitt 6

Innerhalb dieser Methode werden die Python-Skripte nacheinander mit await ausgeführt:

```
await RunPythonAsync(_captionScript, $"\"{destImg}\" \"{captionJson}\"");
await RunPythonAsync(_mapScript, $"\"{captionJson}\" \"{mappedJson}\"");
await RunPythonAsync(_musicScript, $"\"{mappedJson}\" \"{outWavBase}\"");
await RunPythonAsync(_analyzeScript, $"{name} {runId}");
```

Listing 75 — MainWindow.xaml.cs Ausschnitt 7

Durch async und await bleibt die Benutzeroberfläche während der Ausführung vollständig bedienbar.

### Ausführung der Python-Skripte

Die eigentliche Prozessausführung ist in RunPythonAsync() gekapselt:

```
private Task RunPythonAsync(string scriptPath, string args)
{
    return Task.Run(() =>
    {
        var psi = new ProcessStartInfo
        {
            FileName = _pythonExe,
            Arguments = $"\"{scriptPath}\" {args}",
            RedirectStandardOutput = true,
            RedirectStandardError = true,
            UseShellExecute = false,
            CreateNoWindow = true,
            WorkingDirectory = _repoRoot
        };

        using var p = Process.Start(psi);
        var stdout = p.StandardOutput.ReadToEnd();
        var stderr = p.StandardError.ReadToEnd();
        p.WaitForExit();

        if (!string.IsNullOrWhiteSpace(stdout)) AppendLog(stdout.Trim());
        if (!string.IsNullOrWhiteSpace(stderr)) AppendLog(stderr.Trim());

        if (p.ExitCode != 0)
            throw new Exception("Python-Skript fehlgeschlagen.");
    });
}
```

Listing 76 — MainWindow.xaml.cs Ausschnitt 8

Hier wird ein externer Python-Prozess gestartet, dessen Standard- und Fehlerausgabe direkt in das Log-Feld der Oberfläche geschrieben wird. Durch Task.Run() erfolgt die Ausführung in einem Hintergrund-Thread, wodurch das UI nicht blockiert wird.

### Dynamische Aktualisierung der Audioauswahl

Nach erfolgreicher Pipeline-Ausführung werden die generierten WAV-Dateien automatisch geladen. Hierfür wird eine ObservableCollection verwendet:

```
public ObservableCollection<string> AudioFiles { get; } = new();
```

Listing 77 — MainWindow.xaml.cs Ausschnitt 9

Die Methode zum Laden der Dateien lautet:

```
private void LoadAudioChoices(string baseName)
{
    AudioFiles.Clear();
    var files = Directory.EnumerateFiles(_outputsDir, $"{baseName}
_v*.wav")
                            .OrderBy(p => p)
                            .ToList();
    foreach (var f in files)
        AudioFiles.Add(f);

    SelectedAudioFile = AudioFiles.FirstOrDefault();
    AppendLog($"Gefundene WAVs: {AudioFiles.Count}");
}
```

Listing 78 — MainWindow.xaml.cs Ausschnitt 10

Eine ObservableCollection informiert das UI automatisch über Änderungen an der Liste, wodurch das Dropdown-Menü unmittelbar aktualisiert wird.

Die Interaktion zwischen UI und Pipeline basiert auf drei Mechanismen:

1. Data Binding über INotifyPropertyChanged zur automatischen Aktualisierung von Status und Log

2. Asynchrone Programmierung mit async/await zur Vermeidung von UI-Blockierungen

3. Auslagerung der Python-Prozesse in Hintergrund-Threads mittels Task.Run()

Durch diese Architektur bleibt die Benutzeroberfläche jederzeit reaktionsfähig, während im Hintergrund mehrere KI-Modelle sequenziell ausgeführt werden. Die UI übernimmt dabei die Rolle einer orchestrierenden Schicht zwischen Benutzerinteraktion und Python-basierter Pipeline.

## 9.5 Wiedergabe, Speicherung und Analyseanzeige

Nach erfolgreicher Ausführung der KI-Pipeline stellt die Anwendung dem Benutzer mehrere weiterführende Interaktionsmöglichkeiten zur Verfügung. Die generierten Musikdateien können direkt innerhalb der Oberfläche abgespielt, lokal gespeichert sowie zusammen mit dem automatisch erzeugten Analysebericht exportiert werden. Diese Funktionen sind vollständig in der Klasse MainWindow implementiert und greifen auf die im data/outputs- und data/results-Verzeichnis erzeugten Dateien zu.

Die generierten Audiodateien werden nach Abschluss der Pipeline automatisch erfasst und dem Benutzer zur Auswahl bereitgestellt. Hierfür wird eine `ObservableCollection<string>` verwendet wie im Kapitel 9.4 bereits erklärt.

Die eigentliche Audiowiedergabe erfolgt über die Klasse MediaPlayer, die einmalig als Feld initialisiert wird:

```
private readonly MediaPlayer _player = new();
```

Listing 79 — MainWindow.xaml.cs Ausschnitt 11

Beim Klick auf den Play-Button wird geprüft, ob eine gültige Datei ausgewählt wurde. Anschließend wird die Datei geladen und abgespielt:

```
private void Play_Click(object sender, RoutedEventArgs e)
{
    if (string.IsNullOrWhiteSpace(SelectedAudioFile) || !
File.Exists(SelectedAudioFile))
    {
        MessageBox.Show("Keine WAV ausgewählt/gefunden.");
        return;
    }

    _player.Open(new Uri(SelectedAudioFile));
    _player.Play();
    AppendLog("Play: " + SelectedAudioFile);
}
```

Listing 80 — MainWindow.xaml.cs Ausschnitt 12

Die Trennung zwischen Auswahlprüfung, Player-Initialisierung und Logging sorgt für eine klare Struktur. Zusätzlich wird jeder Wiedergabevorgang im Log-Feld dokumentiert, wodurch

der Ablauf für den Benutzer transparent bleibt. Das Stoppen der Wiedergabe erfolgt analog über:

```
private void Stop_Click(object sender, RoutedEventArgs e)
{
    _player.Stop();
    AppendLog("Stop");
}
```

Listing 81 — MainWindow.xaml.cs Ausschnitt 13

Neben der direkten Wiedergabe bietet die Anwendung die Möglichkeit, eine ausgewählte Musikdatei lokal zu speichern. Hierfür wird ein SaveFileDialog verwendet, der dem Benutzer die freie Wahl des Speicherorts erlaubt:

```
private void SaveWav_Click(object sender, RoutedEventArgs e)
{
    if (string.IsNullOrWhiteSpace(SelectedAudioFile) || !
File.Exists(SelectedAudioFile))
    {
        MessageBox.Show("Keine WAV ausgewählt.");
        return;
    }

    var dlg = new SaveFileDialog
    {
        Filter = "WAV|*.wav",
        FileName = Path.GetFileName(SelectedAudioFile)
    };

    if (dlg.ShowDialog() == true)
    {
        File.Copy(SelectedAudioFile, dlg.FileName, overwrite: true);
        AppendLog("WAV gespeichert: " + dlg.FileName);
    }
}
```

Listing 82 — MainWindow.xaml.cs Ausschnitt 14

Durch das Setzen von overwrite: true wird sichergestellt, dass bestehende Dateien bei Bedarf überschrieben werden können. Auch dieser Vorgang wird im Log protokolliert, wodurch eine konsistente Rückmeldung an den Benutzer gewährleistet wird.

Zusätzlich zur Musikdatei erzeugt die Pipeline einen Analysebericht im Markdown-Format, der einen Vergleich der generierten Versionen enthält. Der Pfad zur zuletzt erzeugten Report-Datei wird nach Abschluss der Pipeline gespeichert:

```
private string? _lastReportPath;
```

Listing 83 — MainWindow.xaml.cs Ausschnitt 15

Der Export des Analyseberichts erfolgt über einen separaten Button-Handler:

```
private void SaveAnalysis_Click(object sender, RoutedEventArgs e)
{
    if (string.IsNullOrWhiteSpace(_lastReportPath) || !File.Exists(_lastReportPath))
    {
        MessageBox.Show("Report nicht gefunden. Erst Pipeline laufen lassen.");
        return;
    }

    var dlg = new Microsoft.Win32.SaveFileDialog
    {
        Filter = "Markdown|*.md",
        FileName = Path.GetFileName(_lastReportPath)
    };

    if (dlg.ShowDialog() == true)
    {
        File.Copy(_lastReportPath, dlg.FileName, overwrite: true);
        AppendLog("Report gespeichert: " + dlg.FileName);
    }
}
```

Listing 84 — MainWindow.xaml.cs Ausschnitt 16

Auch hier wird zunächst validiert, ob die Datei tatsächlich existiert. Dadurch wird verhindert, dass der Benutzer versucht, einen Report zu exportieren, bevor die Pipeline vollständig ausgeführt wurde. Das gewählte Dateiformat .md ermöglicht eine einfache Weiterverarbeitung oder Konvertierung in andere Formate wie PDF.

Insgesamt zeigt dieser Abschnitt, dass die Benutzeroberfläche nicht nur als reiner Starter der Pipeline fungiert, sondern als vollständige Interaktionsschicht für Wiedergabe, Dateiverwaltung und Analysebereitstellung dient. Durch die Kombination aus ObservableCollection, MediaPlayer, Dialogfenstern und konsistenter Log-Ausgabe entsteht eine geschlossene

Benutzererfahrung, bei der Generierung, Analyse und Nutzung der Musik nahtlos ineinandergreifen.

# 10 Erkenntnisse

## 10.1 Technische Probleme und Lösungen

Während der Entwicklung des Systems traten verschiedene technische Herausforderungen auf, die sowohl die KI-basierten Komponenten als auch die klassische Softwarearchitektur betrafen. Da mehrere Programmiersprachen, externe Modelle und unterschiedliche Laufzeitumgebungen miteinander kombiniert werden, zeigte sich früh, dass Stabilität und saubere Schnittstellen entscheidend für den Gesamterfolg des Projekts sind.

Problematisch war vor allem die hohe Rechenbelastung durch die eingesetzten KI-Modelle. Insbesondere die Musikgenerierung sowie das Bild-Captioning führten auf leistungsschwächerer Hardware zu langen Laufzeiten. Dies wirkte sich unmittelbar auf die Benutzererfahrung aus. Um diesem Problem zu begegnen, wurden bewusst geeignete Modellgrößen gewählt und die Pipeline asynchron ausgeführt, sodass die Benutzeroberfläche während der Verarbeitung reaktionsfähig blieb. Dadurch konnte trotz längerer Berechnungszeiten eine stabile und nutzbare Anwendung gewährleistet werden. Gleichzeitig wurde ein bewusster Kompromiss zwischen Ergebnisqualität und Performance getroffen.

Eine weitere Herausforderung bestand in der Kommunikation zwischen `C#` und Python. Da die Python-Skripte als externe Prozesse gestartet werden, mussten Parameter, Dateipfade und Ausgabeformate exakt abgestimmt sein. Fehler in der Pfadverwaltung führten anfänglich dazu, dass Skripte keine Ergebnisse erzeugten oder Ausgabedateien nicht gefunden wurden. Gelöst wurde das Problem durch eine feste Ordnerstruktur (data/images, data/results, data/outputs) und eine gebündelte Pfadverwaltung in der UI-Anwendung. Dadurch entstand eine konsistente Schnittstelle zwischen den Systemkomponenten.

Auch die Fehlersuche erwies sich zunächst als schwierig, da bei komplexen Abläufen nicht sofort ersichtlich war, an welcher Stelle ein Fehler auftrat. Die Einführung eines strukturierten Loggings stellte hier einen entscheidenden Fortschritt dar. Durch das systematische Erfassen von Standard- und Fehlerausgaben der Python-Prozesse sowie Statusmeldungen in der Benut-

zeroberfläche konnte der Ablauf der Pipeline transparent gemacht werden. Fehlerquellen ließen sich dadurch deutlich schneller identifizieren und beheben.

Insgesamt zeigte sich, dass die größte Herausforderung nicht in der Implementierung einzelner Funktionen lag, sondern in der Integration und Stabilisierung des Gesamtsystems. Durch asynchrone Verarbeitung, konsistente Schnittstellen und strukturiertes Logging konnte die Architektur schrittweise robuster gestaltet werden. Die Auseinandersetzung mit diesen Problemen trug wesentlich dazu bei, die Zuverlässigkeit und Wartbarkeit des finalen Systems zu erhöhen.

## 10.2 Analyse der WAV-Varianten

Neue Einsichten ergaben sich vor allem aus der systematischen Analyse der drei je Bild generierten WAV-Dateien. Obwohl alle Varianten auf derselben Caption und denselben abgeleiteten Musikparametern basieren, zeigen die extrahierten Audiofeatures deutliche Unterschiede. Zur objektiven Bewertung wurden unter anderem RMS (Energie), spektraler Schwerpunkt (klangliche Helligkeit) sowie ein geschätztes Tempo analysiert.

Die Untersuchung machte deutlich, dass die Varianten trotz identischer Ausgangsbedingungen keine bloßen Duplikate darstellen. Vielmehr unterscheiden sie sich messbar in ihrer Energieverteilung, spektralen Struktur und rhythmischen Ausprägung. Unterschiede im RMS-Wert weisen auf variierende Dynamik- und Lautheitscharakteristika hin, während Abweichungen im spektralen Schwerpunkt auf unterschiedliche klangliche Gewichtungen und Instrumentenanteile schließen lassen. Auch die Tempo-Schätzung zeigte, dass die tatsächlich wahrnehmbare Geschwindigkeit nicht zwingend exakt dem vorgegebenen Zieltempo entspricht.

Daraus ergibt sich eine wesentliche Erkenntnis über das Verhalten des generativen Modells: Die musikalischen Parameter wie Stimmung, Genre oder Tempo definieren einen stilistischen Rahmen, innerhalb dessen das Modell eigenständige klangliche Realisierungen erzeugt. Die Generierung ist somit kontrolliert, jedoch nicht deterministisch. Selbst bei identischen Eingaben entstehen unterschiedliche akustische Ausprägungen, die sich quantitativ erfassen lassen.

Die Analyse bestätigt damit, dass die Mehrfachgenerierung pro Bild kein redundanter Schritt ist, sondern notwendig, um die Variabilität des Systems sichtbar zu machen. Erst durch den

direkten Vergleich der Varianten wird deutlich, in welchen Dimensionen das Modell variiert und wie stark sich einzelne Eigenschaften zwischen den Versionen unterscheiden. Die Kombination aus objektiver Feature-Extraktion und vergleichender Auswertung ermöglicht somit eine wissenschaftlich fundierte Betrachtung des Modellverhaltens über eine rein subjektive Höranalyse hinaus.

Insgesamt führte die Analyse der WAV-Varianten zu der Erkenntnis, dass generative Musikmodelle konsistente stilistische Tendenzen erzeugen, dabei jedoch einen messbaren kreativen Spielraum besitzen. Genau diese Balance zwischen Kontrolle und Variation bildet den Kern des untersuchten Systems.

## 10.3 Ergebnisse aus Beispieldurchläufen

Anschließend werden die konkreten Beispiel-Outputs für fünf Eingabefälle dokumentiert. Der Fokus liegt dabei auf den nachvollziehbaren Zwischenresultaten innerhalb der Pipeline (Bild-Input, Caption, Mapping/Steuerparameter und Analyse-Highlights) sowie auf den erzeugten Audio-Dateien.

### 10.3.1 Inputbilder

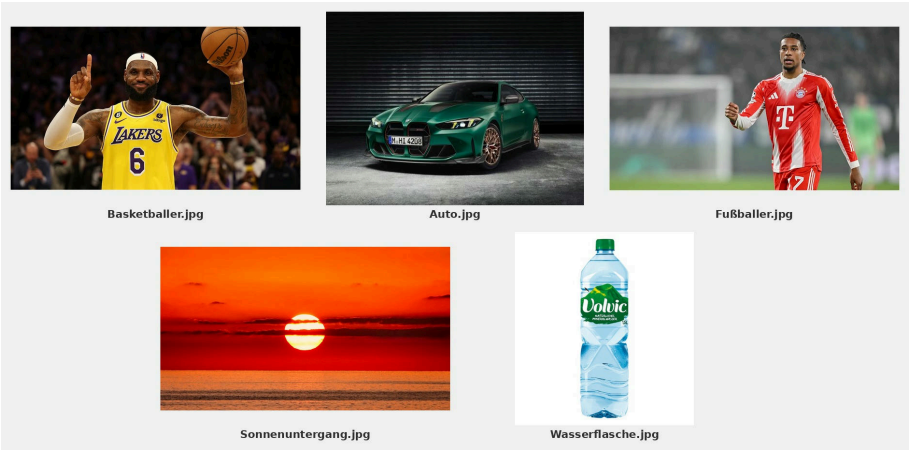


Abbildung 4 — Die Input Bilder

Die fünf ausgewählten Eingabebilder decken bewusst ein breites thematisches Spektrum ab: von einem dynamischen Sportereignis über ein technisches Objekt bis hin zu einer ruhigen Naturszene und einem neutralen Produktbild. Diese Vielfalt dient dazu, die Generalisierungsfähigkeit der Pipeline zu demonstrieren, also die Fähigkeit des Systems, unterschiedlichste

visuelle Inhalte in musikalisch sinnvolle Ausgaben zu überführen. Gleichzeitig erlaubt die Auswahl einen impliziten Vergleich zwischen Bildern, die intuitiv sehr unterschiedliche emotionale Assoziationen wecken, und solchen, bei denen eine musikalische Interpretation weniger naheliegend erscheint.

### 10.3.2 Zwischenergebnis 1: Caption

Die Captions stammen aus `data/results/<name>.json` (Feld `caption`):

- **Auto**: a green sports car parked in front of a garage (Quelle: `data/results/Auto.json`)
- **Basketballer**: a man in a yellow jersey holding a basketball ball (Quelle: `data/results/Basketballer.json`)
- **Fußballer**: a soccer player in a red and white uniform on a field (Quelle: `data/results/Fußballer.json`)
- **Sonnenuntergang**: the sun is setting over the ocean (Quelle: `data/results/Sonnenuntergang.json`)
- **Wasserflasche**: a bottle of water on a white background (Quelle: `data/results/Wasserflasche.json`)

Die erzeugten Bildbeschreibungen sind knapp, sachlich und auf die visuell auffälligsten Merkmale des jeweiligen Bildes fokussiert. Sie spiegeln das in Kapitel 2.2 beschriebene Verhalten von Image-Captioning-Modellen wider: Das BLIP-Modell liefert keine erschöpfende Beschreibung aller Bilddetails, sondern eine semantisch verdichtete Zusammenfassung, die den Kern des Bildinhalts in natürlicher Sprache erfasst. Auffällig ist, dass die Beschreibungen keine expliziten emotionalen Aussagen enthalten, der Sonnenuntergang wird als rein visuelles Ereignis beschrieben, ohne dass Stimmungsbegriffe wie „friedlich„ oder „melancholisch“ auftauchen. Die emotionale Interpretation bleibt damit vollständig dem nachfolgenden Mapping-Schritt überlassen, was die in Kapitel 5.1 beschriebene Notwendigkeit eines hybriden Ansatzes unterstreicht.

### 10.3.3 Zwischenergebnis 2: Mapping / Musikparameter

Die Mapping-Ergebnisse (Stimmung, Genre, Tempo und Instrumente) stammen aus `data/results/<name>_mapped.json`:

- **Auto**: mood `energetic`, genre `pop`, tempo `130 bpm`, instruments `drums, bass, keys` (Quelle: `data/results/Auto_mapped.json`)

- **Basketballer**: mood `dramatic`, genre `lofi`, tempo `85 bpm`, instruments `vinyl noise, piano, soft drums` (Quelle: `data/results/Basketballer_mapped.json`)
- **Fußballer**: mood `dramatic`, genre `lofi`, tempo `85 bpm`, instruments `vinyl noise, piano, soft drums` (Quelle: `data/results/Fußballer_mapped.json`)
- **Sonnenuntergang**: mood `dark`, genre `cinematic`, tempo `75 bpm`, instruments `strings, pads, low brass` (Quelle: `data/results/Sonnenuntergang_mapped.json`)
- **Wasserflasche**: mood `calm`, genre `ambient`, tempo `75 bpm`, instruments `pads, soft synth` (Quelle: `data/results/Wasserflasche_mapped.json`)

Die Mapping-Ergebnisse zeigen, dass der in Kapitel 5 beschriebene hybride Ansatz in den meisten Fällen plausible Zuordnungen produziert. Das Bild des Sportwagens wird als energetisch eingestuft und erhält ein hohes Tempo von 130 BPM, was der visuellen Dynamik des Motivs entspricht. Der ruhige Sonnenuntergang wird als dunkel-cinematisch interpretiert, was angesichts der tiefen Farbgebung des Bildes nachvollziehbar ist. Bemerkenswert ist die identische Zuordnung für Basketballer und Fußballer, beide werden als dramatic-lofi mit 85 BPM klassifiziert. Dies verdeutlicht eine der in Kapitel 1.3 benannten Grenzen des Systems: Das Zero-Shot-Modell reagiert auf die semantische Oberfläche der Caption, ohne tiefergehende kontextuelle Unterschiede zwischen den Sportarten zu berücksichtigen. Das Produktbild der Wasserflasche erhält erwartungsgemäß die neutralste Zuordnung, was zeigt, dass das System auch bei visuell wenig emotionalem Inhalt stabile Ergebnisse liefert.

### 10.3.4 Zwischenergebnis 3: Analyse-Highlights

Die Analyse-Highlights stammen aus `data/results/comparison_report_<name>_...md`:

- **Auto**: Loudest v1, Brightest v3, Fastest v1; RMS range `0.0215`, Centroid range `1439.7 Hz`, Tempo range `0.0 BPM` (Quelle: `data/results/comparison_report_Auto.md`)
- **Basketballer**: Loudest v3, Brightest v2, Fastest v1; RMS range `0.1255`, Centroid range `771.0 Hz`, Tempo range `85.2 BPM` (Quelle: `data/results/comparison_report_Basketballer.md`)
- **Fußballer**: Loudest v1, Brightest v3, Fastest v2; RMS range `0.1213`, Centroid range `2163.8 Hz`, Tempo range `26.9 BPM` (Quelle: `data/results/comparison_report_Fußballer.md`)
- **Sonnenuntergang**: Loudest v3, Brightest v2, Fastest v1; RMS range `0.0514`, Centroid range `750.5 Hz`, Tempo range `37.1 BPM` (Quelle: `data/results/comparison_report_Sonnenuntergang.md`)

- **Wasserflasche**: Loudest v2, Brightest v1, Fastest v1; RMS range `0.0739`, Centroid range `272.9 Hz`, Tempo range `132.8 BPM` (Quelle: `data/results/comparison_report_Wasserflasche.md`)

Die Analyse-Highlights bestätigen die in Kapitel 10.2 gewonnene Erkenntnis, dass die drei pro Bild erzeugten Musikversionen trotz identischer Steuerparameter messbar voneinander abweichen. Besonders deutlich zeigt sich dies beim Basketballer und Fußballer, wo RMS-Ranges von über 0.12 und erhebliche Centroid-Spannen auf eine ausgeprägte interne Variabilität des generativen Modells hinweisen. Im Kontrast dazu fällt die Auto-Variante mit einer RMS-Range von nur 0.0215 und einer Tempo-Range von 0.0 BPM deutlich stabiler aus, was auf eine konsistentere Generierung bei energetisch-rhythmischen Steuerparametern hindeutet. Die Wasserflasche zeigt mit einer Tempo-Range von 132.8 BPM eine besonders hohe rhythmische Variabilität, obwohl das Mapping ruhige Ambient-Musik vorgegeben hatte, ein Hinweis darauf, dass das Modell bei abstrakten, schwach differenzierten Eingaben einen größeren kreativen Spielraum ausschöpft. Insgesamt bestätigen diese Ergebnisse, dass die quantitative Audioanalyse ein notwendiges Werkzeug ist, um das Verhalten des Systems über den subjektiven Höreindruck hinaus objektiv zu beschreiben.

### 10.3.5 Audio-Outputs

Die erzeugten Audio-Dateien liegen unter `data/outputs/`:

- **Auto**: `Auto_v1.wav`, `Auto_v2.wav`, `Auto_v3.wav`
- **Basketballer**: `Basketballer_v1.wav`, `Basketballer_v2.wav`, `Basketballer_v3.wav`
- **Fußballer**: `Fußballer_v1.wav`, `Fußballer_v2.wav`, `Fußballer_v3.wav`
- **Sonnenuntergang**: `Sonnenuntergang_v1.wav`, `Sonnenuntergang_v2.wav`, `Sonnenuntergang_v3.wav`
- **Wasserflasche**: `Wasserflasche_v1.wav`, `Wasserflasche_v2.wav`, `Wasserflasche_v3.wav`

Die erzeugten WAV-Dateien stellen das finale Ergebnis der Pipeline dar und bilden zugleich die Grundlage für die in Kapitel 7 beschriebene Audioanalyse. Jede Datei ist direkt über die in Kapitel 9 entwickelte WPF-Oberfläche abspielbar, was eine unmittelbare Evaluation der generierten Musik im Kontext des Eingabebildes ermöglicht. Die Tatsache, dass pro Bild drei Versionen vorliegen, erlaubt nicht nur einen Vergleich innerhalb eines Motivs, sondern auch eine Einschätzung, wie stabil oder variabel das System unter identischen Ausgangsbedingun-

gen agiert. In der Gesamtbetrachtung der fünf Eingabefälle zeigt sich, dass die Pipeline für alle untersuchten Motivtypen vollständig und fehlerfrei durchläuft, ein Ergebnis, das die technische Robustheit des Systems unterstreicht und eine solide Grundlage für die im folgenden Kapitel formulierten Schlussfolgerungen und Erweiterungsvorschläge bildet.

## 10.4 Fazit und Ausblick

Die vorliegende Arbeit hat gezeigt, dass eine vollautomatisierte Pipeline zur Generierung von Musik aus Bildern mithilfe aktueller KI-Modelle realisierbar ist. Durch die Kombination von BLIP für die Bildbeschreibung, BART für die Zero-Shot-Klassifikation und MusicGen für die Musikerzeugung konnte ein System entwickelt werden, das ohne manuelle Zwischenschritte von einem Eingabebild zu einem hörbaren Musikstück gelangt. Die Orchestrierung über `C#` und .NET 10 ermöglicht dabei eine klare Trennung zwischen Steuerungslogik und KI-Verarbeitung, während die WPF-Oberfläche eine benutzerfreundliche Bedienung gewährleistet.

Zielerreichung. Die zu Beginn der Arbeit formulierten Ziele wurden erreicht: Das System nimmt ein beliebiges Bild entgegen, erzeugt automatisch eine textuelle Beschreibung, leitet daraus Stimmung und Genre ab und generiert darauf basierend drei Varianten eines Musikstücks. Die anschließende Audio-Analyse mittels librosa liefert objektive Kennzahlen zum Vergleich der erzeugten Versionen.

Limitationen. Trotz der funktionierenden Umsetzung bestehen Einschränkungen, die nicht verschwiegen werden sollen. Die Qualität der generierten Musik ist durch die Verwendung des kleinsten MusicGen-Modells begrenzt. Die erzeugten Stücke sind relativ kurz und eignen sich eher als Klangskizzen denn als vollständige Kompositionen. Das regelbasierte Mapping von Stimmungen auf Tempo und Instrumentierung ist deterministisch und bildet die Komplexität musikalischer Gestaltung nur vereinfacht ab. Zudem ist die Pipeline rechenintensiv und erfordert bei erstmaliger Ausführung den Download mehrerer großer Modelle.

Ausblick. Für eine Weiterentwicklung bieten sich mehrere Ansatzpunkte an. Der Einsatz größerer MusicGen-Varianten könnte die musikalische Qualität und Länge der Ausgabe deutlich verbessern. Das regelbasierte Mapping ließe sich durch ein trainiertes Modell ersetzen, das den Zusammenhang zwischen visuellen Merkmalen und musikalischen Parametern aus Daten lernt. Eine Echtzeit-Vorschau in der Benutzeroberfläche sowie die Möglichkeit, generierte Stücke interaktiv anzupassen, würden den praktischen Nutzen des Systems erhöhen. Darüber

hinaus wäre eine systematische Evaluation mit Nutzerstudien sinnvoll, um die subjektive Passgenauigkeit zwischen Bild und Musik zu bewerten.

# Literatur


[1] J. Copet *u. a.*, „Simple and Controllable Music Generation“, 2023, [Online]. Verfügbar unter: https://arxiv.org/pdf/2306.05284.pdf

[2] Microsoft Corporation, „C# Guide - .NET managed language“. [Online]. Verfügbar unter: https://learn.microsoft.com/en-us/dotnet/csharp/

[3] D. Crockford, „The application/json Media Type for JavaScript Object Notation (JSON)“. [Online]. Verfügbar unter: https://datatracker.ietf.org/doc/html/rfc4627

[4] J. Li, D. Li, C. Xiong, und S. C. Hoi, „BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation“, 2022, [Online]. Verfügbar unter: https://arxiv.org/abs/2201.12086

[5] T. Wolf *u. a.*, „HuggingFace's Transformers: State-of-the-art Natural Language Processing“, 2020, [Online]. Verfügbar unter: https://arxiv.org/abs/1910.03771

[6] Pillow Contributors, „Pillow (PIL Fork) Documentation“. [Online]. Verfügbar unter: https://pillow.readthedocs.io/en/stable/

[7] A. Paszke *u. a.*, „Automatic differentiation in PyTorch“, 2017. [Online]. Verfügbar unter: https://openreview.net/forum?id=BJJsrmfCZ

[8] PyTorch Team, „PyTorch Documentation“. [Online]. Verfügbar unter: https://pytorch.org/docs/stable/index.html

[9] M. Palatucci, D. Pomerleau, G. E. Hinton, und T. M. Mitchell, „Zero-shot Learning with Semantic Output Codes“, 2009. [Online]. Verfügbar unter: http://papers.nips.cc/paper/3650-zero-shot-learning-with-semantic-output-codes.pdf

[10] M. Lewis *u. a.*, „BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension“, 2019, [Online]. Verfügbar unter: https://arxiv.org/abs/1910.13461

[11] Meta AI, „AudioCraft - Generative AI for audio and music“. [Online]. Verfügbar unter: https://ai.meta.com/resources/models-and-libraries/audiocraft/

[12] C. R. Harris *u. a.*, „Array programming with NumPy“, *Nature*, 2020, [Online]. Verfügbar unter: https://www.nature.com/articles/s41586-020-2649-2

[13] P. Virtanen *u. a.*, „SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python“, 2020, [Online]. Verfügbar unter: https://arxiv.org/abs/1907.10121

[14] SciPy Community, „SciPy API Manual (v1.x)“. [Online]. Verfügbar unter: https://docs.scipy.org/doc/scipy/reference/

[15] B. McFee *u. a.*, „librosa: Audio and Music Signal Analysis in Python“, 2015. [Online]. Verfügbar unter: https://brianmcfee.net/papers/scipy2015_librosa.pdf

[16] S. B. Davis und P. Mermelstein, „Comparison of Parametric Representations for Monosyllabic Word Recognition in Continuously Spoken Sentences“, *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 1980, [Online]. Verfügbar unter: https://courses.grainger.illinois.edu/ece417/fa2017/davis80.pdf

[17] Microsoft Corporation, „Windows Presentation Foundation (WPF) Documentation“. [Online]. Verfügbar unter: https://learn.microsoft.com/en-us/dotnet/desktop/wpf/